

Continual Field-Adaptive Models (CFAMs) for Post-Deployment Physical AI

Autonomous Few-Shot Continual On-Device Adaptation Post-Deployment

Amarjot Singh¹ Tanmay R. Pancholi¹ Jainam Kothari¹
 Shrirang Mahajan¹ Ketan Bansal¹ Zackory Erickson²
 Giuseppe Loianno³ Alexandre M. Bayen³ Jeff Schneider² Vince Nakayama¹
¹Skylark Labs ²Carnegie Mellon University ³University of California, Berkeley

Abstract

Unattended interactive autonomy—machines that step into danger in place of humans and complete tasks with the very tools humans use—is the missing capability in mission-critical operations. The domains that need it offer scarce training data and only the compute the asset carries, yet the field keeps presenting novelty that a deployed model cannot learn without erasing what it already knows.

We introduce **Continual Field-Adaptive Models (CFAMs)**, a model class built for this regime: it learns efficiently in the lab, and then keeps learning after deployment through autonomous, gradient-free, on-device updates. A CFAM is a brain-inspired complementary learning system. Its slow-learning part, frozen after the lab in the role of the slow-learning cortical component of complementary learning systems, is three cortices: a Sensor cortex that lifts multimodal input into 3D-grounded geometry, a Reasoning cortex that decomposes tasks into skills and judges their outcomes, and an Action cortex, a geometric skill model, that executes each skill. Its fast-learning part is the Capsule Field, a hippocampus-like memory where all field learning is written one-shot and gradient-free as Competence Capsules, one capsule per stored competence element. Skill installation is therefore few-shot in the lab on top of the pretrained prior and continual in the field; open-world novelty is outside its scope.

We validate CFAM across five embodiments (manipulator, quadruped, humanoid, quadrotor, and off-road vehicle), with every baseline policy (π_0 , CogACT, SpatialVLA) trained on the same in-house multi-embodiment dataset for the physical-platform comparisons. On the training side, CFAM is data-efficient: it reaches the operating point of the standard policy trained on the full prior-training dataset while using only 40% of that data ($2.5\times$ fewer prior-training trajectories). At test time, it grows: autonomous capture of verified near-OOD cases raises action success by 13.9 percentage points while adaptation baselines fall short. And, in the sequential simulation suite, earlier competence is retained: backward transfer is -0.5 percentage points, versus -11.4 percentage points for LoRA. Together these give a bounded form of post-deployment physical intelligence: learn a task few-shot in the lab, keep growing it autonomously after deployment from verified, slightly out-of-distribution experience, and keep what was gained.

1 Introduction

Autonomy in defense and other mission-critical domains has so far relied on two systems. Surveillance systems observe and report, and one-shot strike systems deliver a single pre-committed action. Neither system interacts with or adapts to the world it operates in. Yet mines, seaports, battlefields, and public-safety operations are full of tasks that demand exactly this interaction, and today every one of them puts a human in harm’s way. A suspected explosive device must still be cleared by hand, a shaft graded for imminent collapse must still be walked by an inspector, and an unstable structure after an earthquake must still be searched by the people who climb into it. The missing capability is *unattended interactive autonomy*, machines that step into danger in place of humans and complete their tasks with the very tools humans use.

The same missing capability also keeps many new applications out of reach entirely. Beyond the tasks that endanger humans lie missions for which no training data exists and none

can be collected in advance: the hadal ocean below 6,000 meters, the Martian surface with its 4–24 minute communication delay, radiation-saturated reactor interiors. The hardware, sensors, and control algorithms for these missions exist; the data to train a model for them does not, so a machine can enter such a world only if it can adapt once it is there.

Such systems are difficult to build due to three key challenges:

- (i) *Data*. Modern robot-learning models acquire their competence from thousands of hours of demonstrations collected through instrumented, internet-scale pipelines [1, 2]; mission-critical operations offer no such pipeline (their environments are hazardous to instrument, access to them is restricted, and much of what they record is classified), so the demonstrations that exist number in the handfuls and rarely transfer from one mission to the next.
- (ii) *Compute*. Robot-learning models are built for clusters with reliable connectivity, but there is no data center behind a quadruped in a mine shaft or a drone over a disaster site: the model must run on the edge device the asset

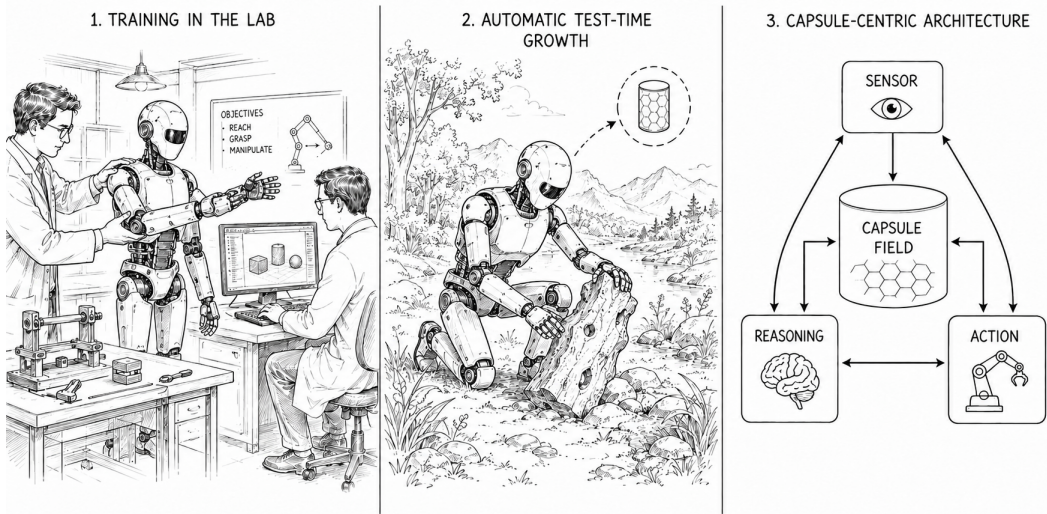


Figure 1: The Continual Field-Adaptive Model (CFAM) lifecycle and architecture. (1) **Few-shot Build:** a handful of operator demonstrations install seed skills as Competence Capsules without retraining the frozen prior. (2) **Autonomous Test-Time Growth:** after deployment, confidence and retrieval distance identify verified successful near-OOD cases, stored by an on-device, one-shot write with no operator label; each capture expands the support envelope, bounded to known skill families. (3) **Capsule Field Architecture:** the Sensor, Reasoning, and Action modules retrieve, execute, and consolidate bounded capsules in a fixed-budget field.

carries, often disconnected for the whole mission; this is achieved not by distilling a large model into a small one but by designing efficient architectures that capture representations of the problem and fit within that device’s compute, memory, and power.

- (iii) *New field cases.* Deployment introduces new object arrangements, loads, terrain conditions, and combinations from the first day in the field. These cases are slightly out of distribution but still close to known skills; a frozen model degrades against them, laboratory retraining is too slow or unavailable, and naive in-place updates can erase earlier knowledge through catastrophic forgetting. Unrelated open-world tasks are outside this paper’s scope.

A model class for this setting must therefore learn from a few examples, run on hardware a machine can carry, and keep learning without forgetting; Section 2 formalizes these conditions as the *field learning regime*, and this paper introduces an architecture built for it.

We present **Continual Field-Adaptive Models (CFAMs)**, a new architecture inspired by the brain, built to give mission-critical Physical AI what it is missing: a model that learns tasks few-shot and then autonomously captures verified, slightly out-of-distribution experience on-device after it leaves the lab (Figure 1). The unit of this learning is the **Competence Capsule (CC)**: a compact, callable record of one skill that jointly captures the perception and action information bound to the situation that activates it, and from which the system generalizes—one capsule, learned from as little as one example, is warped geometrically onto new scenes rather than retrained. The design mirrors complementary learning in the brain [3]: frozen substrates provide slow, stable competence in the role of the slowly learning *neocortex*, while a new-learning layer rapidly encodes new episodes like the *hippocampus* and consolidates them over time.

Concretely, a CFAM is a custom VLA stack with two learning phases. *In the lab*, it learns efficiently: a **Sensor module** [4, 5, 6]

lifts multi-modal inputs into 3D-grounded geometry, a **Reasoning module** (a vision-language model custom-trained for mission-critical operations) decomposes tasks into skills and judges their outcomes, and an **Action module** executes each skill as one geometrically warped emission onto the current 3D scene via the Geometric Residual Transform (GRT), so a handful of demonstrations per task, not thousands, install the skill library on top of the pretrained prior. *In the field*, it keeps learning: the **new-learning layer**, governed by the Continual Field Update Rule (CFUR, Section 6.1), writes new competence on the edge device, autonomously, with no operator label and no gradient step; each write is one forward pass and one memory insertion, designed to fit the control cycle. The capturable events are **near-OOD cases** (or **near-edge novelty**): situations slightly out of distribution but close enough that a stored skill still warps into a verified success. Each capture becomes a new capsule, one-shot, within a fixed memory budget and with near-zero interference with what is already stored, so the envelope the library covers widens with use without overwriting what is already stored; no existing adaptation family (gradient-based, reinforcement learning, memory-augmented, or test-time adaptation) offers this combination (Section 3), and the design carries formal deployment guarantees (locality, bounded authority, and convergence), each proved under its stated assumptions in the supplementary material. The Action module and the near-edge extension path of the new-learning layer are the novel contributions of this work, and both are evaluated here; correction from detected field failures and open-world novelty are outside this paper’s scope (Section 8.2), and the Sensor and Reasoning modules are described as designed and integrated, with isolated evaluation in Tables 8 and 9.

Contributions. This paper makes four contributions:

1. **Architecture:** CFAMs, an end-to-end field-adaptive model class for Physical AI, four pieces (Sensor / Reasoning / Action / new-learning layer) built for settings where training

data is limited in the first place. It includes the Action module’s *unit-of-inference* shift (per-skill capsule emissions via GRT, $M \ll T$ action-decoding calls per task) and the new-learning layer’s architectural interface (the capsule schema every field write must produce, and the guarantees imposed on any writer, proved in the supplementary material).

2. **Mission-critical dataset:** Skylark’s in-house multi-embodiment dataset (2.6 million+ trajectories across five physical platforms) on which CFAM and every standard-policy baseline (π_0 , CogACT, SpatialVLA) are trained for the physical-platform comparisons, so those comparisons are matched-data (the simulation benchmarks use the public backbones).
3. **Efficient lab learning:** on the learning curve, CFAM matches the standard policy trained on 100% of D_{train} using only 40% of it, and leads every matched-data baseline on the held-out split (Table 2).
4. **Autonomous post-deployment growth with near-zero measured forgetting:** on the variation stream D_{var} , autonomous test-time capture of new cases raises action success by 13.9 pp (74.0 $\rightarrow$ 87.9, Table 4) while adaptation baselines fall short (Table 5); in the sequential simulation suite retention stays near-perfect, backward transfer -0.5 pp vs. -11.4 pp for LoRA [7] (Table 6).

Paper organization. Section 2 first derives the field-learning requirements from deployment conditions; Section 3 then evaluates prior adaptation families against those requirements and isolates the remaining gap. Section 4 states CFAM’s architectural response; Section 5 presents the four-piece architecture, Section 5.4 defines the Competence Capsule, and Section 5.3 covers how skills are selected, warped, and executed. Section 6 develops test-time growth, the CFUR capture rule, and compression. Section 7 reports the pre-novelty experiments, Section 8 discusses limitations, and Section 9 concludes.

2 The Field Learning Regime: Why a Deployed Model Must Keep Growing

The world a mission presents cannot be captured in any pre-deployment dataset, and in mission-critical domains the pre-deployment data is limited in the first place (Section 1), so the system must learn the world after deployment. But “after deployment” is not the lab: no curator selects training examples, no class distribution is balanced, and the machine is alone with whatever compute it carried in and whatever data the world decides to show it. What the world shows it is a *stream*: events arrive one at a time, drift steadily further from the training distribution as the mission ages, and occasionally present something genuinely new, the shape the evaluation protocol of Section 7.1 makes measurable.

We call these conditions the **field learning regime**, and three properties define it. Connectivity is absent or intermittent. Supervision is episodic rather than continuous, arriving as a human demonstration here or a sensor spike there. And the operational

tempo requires that any on-device write complete within the controller’s cycle time, without interrupting or catastrophically delaying the task the system was deployed to perform.

Field-learning requirements. Any architecture that meets this regime by *growing* rather than retraining must satisfy four requirements:

- **R1 (Efficient learning):** the system must build a usable competence library from a handful of demonstrations, not the thousands a from-scratch policy needs.
- **R2 (Typed growth):** the system must extend competence through distinct mechanisms matched to the gap: *correction* of a familiar mistake and *extension* to a within-family instance, without retraining the base.
- **R3 (Bounded consolidation):** the system must merge redundant competence and hold a fixed memory footprint as it grows, so new writes never corrupt the skills it already has.
- **R4 (Field-feasible growth):** each new capsule must be written in one shot, gradient-free and on-device, within fixed compute and memory and the control-cycle budget.

The current model class satisfies none of them: it is data-hungry at build time (R1) and cannot grow after deployment (R2–R4); Section 3 provides the cited, family-by-family assessment. The core memory unit (Section 5.4) and the architecture built around it (Section 5) are designed to fill both gaps.

The new cases the field presents are concrete: a known grasp with the object farther away or occluded, a familiar carry chain meeting a heavier load, a route learned on grass arriving at dense weeds (Figure 13)—slightly out-of-distribution, within-family variants for which prior geometry and action structure remain usable. A failure is classified by which part of the frozen model’s prior knowledge is intact and which is missing, and before genuine novelty arrives two regimes cover what a deployed model encounters:

- **Familiar mistake $\rightarrow$ Correction:** a known category with the wrong parameter (a grasp overshoot, a wrist rotation off by a few degrees), needing a tight, surgical fix rather than a new skill.
- **Within-family gap $\rightarrow$ Extension:** an unseen member of a known family (the model grasps cups but not thermoses, walks on concrete but not gravel), needing competence extended across related contexts.

Each regime is read off from signals the system already computes (the base policy’s confidence and the Hamming distance from stored competence in the sparse distributed memory [8]), so no external oracle is needed; the capsule’s regime-dependent storage realizes the mapping (Section 5.4). CFAM expands competence within skill families it already holds—a failure with no relevant family is open-world novelty, outside this architecture’s scope (Section 8.2).

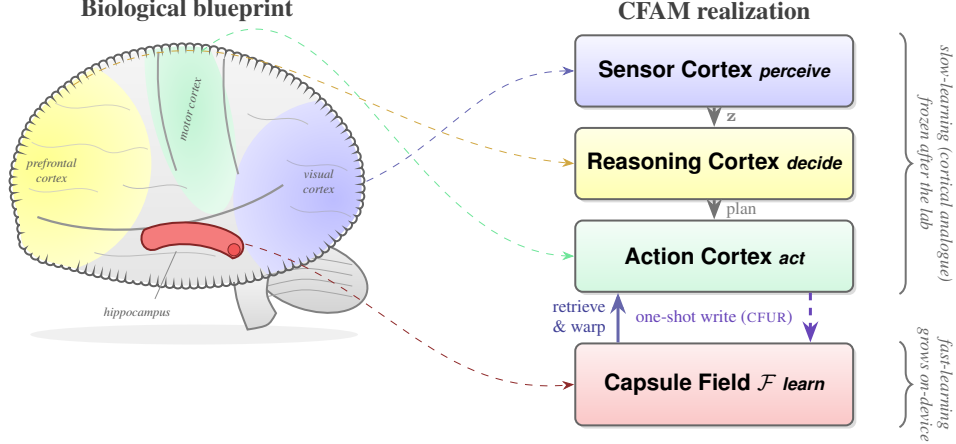


Figure 2: The brain’s learning architecture, transcribed. *Left:* the slow-learning neocortical structures (visual: perceive; prefrontal: decide; motor: act) and the fast-learning hippocampus, which encodes new episodes in one exposure. *Right:* CFAM realizes each structure (dashed mappings): the Sensor, Reasoning, and Action cortices are trained in the lab and frozen at deployment, and the **Capsule Field** $\mathcal{F}$ is the hippocampal memory, written one-shot and gradient-free by the new-learning layer (CFUR), one Competence Capsule per stored competence element. In CFAM the division is strict in both directions: the slow part is frozen after the lab (a deliberate idealization of the cortical side, which in the biological account learns slowly rather than not at all), and the fast part never acts on the world directly.

3 Related Work

Section 2 established four field-learning requirements: efficient few-shot learning (R1), typed correction and extension (R2), bounded consolidation without interference (R3), and one-shot, gradient-free, on-device growth (R4). No prior family supplies the complete operational primitive these define—a closed adaptation object that captures competence at the edge of a frozen model’s envelope, re-applies it during operation, and consolidates it under a fixed budget—so we assess four families against them before stating the architectural gap CFAM addresses (Section 3.5).

3.1 Foundation Policies and Gradient-Free Adaptation

Vision-language-action models (RT-2 [9], OpenVLA [1], Octo [10], CogACT [11], GR00T [12]) collapse perception, reasoning, and action into one model that is *frozen at deployment*; improvement requires centralized retraining, infeasible on edge hardware. CFAM factors the stack (Section 5) and adds the new-learning layer no VLA provides. Among gradient-free alternatives, skill-library and self-critique agents (Voyager [13], ExpEL [14], Reflexion [15], REFLECT [16], HELPER [17], TidyBot [18], Statler [19], AdaPlanner [20]) act on symbolic skills with no physical execution operator, while residual and in-context adapters (Side-Tuning [21], MoS-VLA [22], MAC [23], MAP-VLA [24], TT-VLA [25], EVOLVE-VLA [26], residual policy learning [27, 28]) produce global or gradient-trained residuals, and classical reinforcement learning needs rewards, enumerable states, and replayable trajectories the field does not offer, at per-step value-query cost. Real-world policy improvement through repeated physical execution and feedback has also been explored in ENPIRE [29]. CFAM occupies a third position: physically grounded capsules executed one shot per skill by GRT and sequenced at $O(M)$ emissions (Section 5.3), written

one-shot, prediction-error-gated, and gradient-free. The skill primitive is a synthesis of established ingredients (movement primitives [30], task-parameterized mixtures [31], trajectory transfer [32], keypoint affordances [33], structured skill representations [34]) over a single persistent, consolidating capsule (Section 5.4). Dagger [35] and RMA [36] share the structure but require an expert or privileged teacher; CFAM captures competence from self-observed signals alone (ebbing base confidence, rising retrieval distance) and accumulates explicit residuals over the lifetime.

3.2 Memory-Augmented Policies and Retrieval

Memory-augmented robots organize storage by *cognitive type*: MemoryVLA [37], RoboMemory [38] (after Tulving [39]), EchoVLA [40], and 3DLLM-Mem [41]; lifelong skill-memory and skill-library approaches include ViReSkill [42], LRLL [43], and Uni-Skill [44]. CFAM organizes by *learning function*, corrective and extension regimes with distinct support radii and generalization bandwidths (Section 5.4), and targets runtime growth under fixed memory and constant-time retrieval rather than temporal or spatial reasoning: a different desideratum, not a strict superset. Its substrate extends Kanerva’s sparse distributed memory [8, 45] (cf. Memory Layers at Scale [46], Sparse Memory Finetuning [47]) with confidence-gated blending, one-shot capsule writes, and self-compressing consolidation. One-shot imitation methods (Instant Policy [48], Coarse-to-Fine Imitation [49]) inspire GRT’s geometric reasoning but operate statelessly; CFAM couples one-shot encoding with continual retrieval, so performance improves with experience.

3.3 Test-Time and Continual Learning

Test-time adaptation (TENT [50], TTT [51], CoTTA [52], MEMO [53]) and parameter-efficient adapters (VPT [54], IA³ [55]) adapt backbones by back-propagating through them,

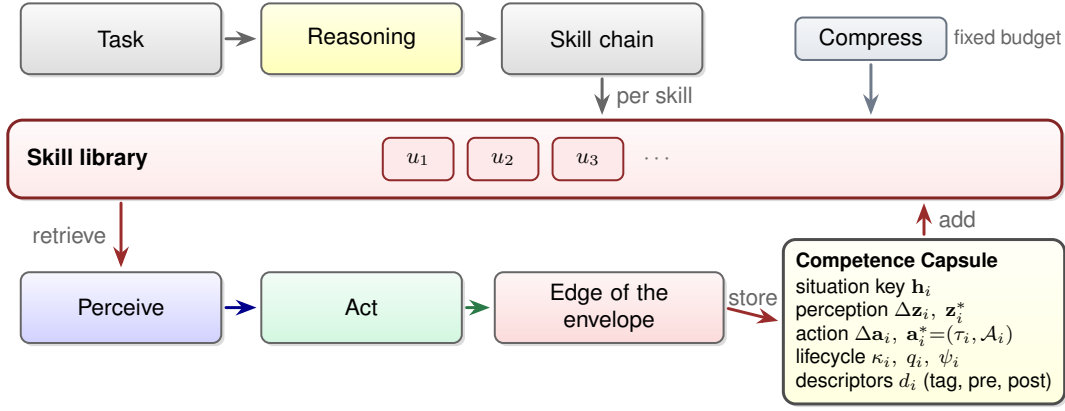


Figure 3: How a CFAM runs a task, and how its library grows. The Reasoning cortex decomposes the task into a chain of skills; for each skill the system perceives the scene in 3D, retrieves the matching capsule, and warps it onto that scene to act—no gradient step, no per-step decoding. An execution that succeeds at the edge of a skill’s envelope is stored as a new **Competence Capsule** (Equation (4)), so the library grows with use, while decay and consolidation (*Compress*) hold it within a fixed memory budget. Colors follow Figure 2.

transiently, on unsupervised signals; CFAM uses no backward pass, gates writes on the base’s own confidence and retrieval distance, and accumulates persistent capsules routed to typed memory regimes. Classical continual learning, regularization (EWC [56]), dynamic architectures [57], replay, and meta-learning (MAML [58]), pursued under programs such as DARPA L2M [59], assumes training-time access to the task stream and gradient updates, both absent on a deployed edge device (Section 2). Closest in structure are backbone prototype methods, NCM and incremental variants [60, 61, 62], which mitigate forgetting by construction through additive per-class enrollment; CFAM is their physical-action generalization, replacing the per-class centroid with a typed, radius-addressed, geometrically warpable capsule that both corrects and grows (Section 7.4). Both families’ protocols announce task boundaries or score stationary test sets; the deployment-stream protocol of Section 7.1 measures what they hold fixed, whether performance rises while the evaluation itself gets harder.

3.4 Perception and Neuroscience Grounding

The Sensor cortex’s SHDL [63, 4] sits among hybrid parametric encoders rather than the monolithic deep encoders that dominate VLA work (ViT [64] and kin), trading end-to-end capacity for fixed front-end invariances, label efficiency, and continual category acquisition [5, 65]; the Scattering Vision Transformer [66] externally validates the pattern at transformer scale. The capsule’s prediction-error lifecycle echoes, but was not reverse-engineered from, predictive coding and the free energy principle [67, 68, 69], complementary learning systems [3, 70], population coding [71, 72], and salience-gated consolidation [73, 74, 75]; the mapping is collected in the supplementary neuroscience-grounding appendix.

3.5 Requirement-Level Gap and CFAM’s Response

The comparison is consistent across families. Foundation policies provide broad initial competence but require centralized gradient retraining, missing R1 and R4. Symbolic skill libraries and retrieval memories can add information without changing

the base, but they do not type correction versus extension or maintain a bounded, non-interfering physical competence store, missing R2 and R3. Test-time and continual-learning methods update parameters or transient state and therefore miss persistent, one-shot field growth under the edge budget, R3–R4. Prototype methods preserve old entries, but do not encode or geometrically execute physical skills.

CFAM is designed as the response to this requirement-level gap: a frozen slow-learning stack for broad competence (R1), typed Competence Capsules for corrective and extension writes (R2), a capsule field that consolidates them under a fixed budget (R3), and a one-shot, back-propagation-free write path (R4). The next section derives that architecture.

4 A Brain-Inspired Architecture for Continual Learning

The field learning regime of Section 2 poses a structural question before it poses an engineering one: *what shape must a model have to learn from single field events without erasing the competence it already holds?* We address this stability–plasticity trade-off through explicit architectural separation rather than within one set of weights, which would have to be simultaneously plastic enough to absorb a one-shot event at the edge and stable enough to protect everything learned before it, the dilemma that per-weight remedies (regularization, replay, parameter isolation) mitigate but do not remove (Section 3). The resolution the brain arrives at is architectural, not parametric: split learning across *two* subsystems that learn at different rates, a **complementary learning system** [3, 70, 76].

CFAM’s architecture is inspired directly by the brain: it adopts this complementary-learning formulation as its design principle (Figure 2). A **slow-learning phase** acquires broad competence efficiently in the laboratory (where data, compute, and supervision are plentiful) and is then frozen, playing the role of the slow-learning cortical component of a complementary learning system (an idealization: biological cortex learns slowly rather than not at all), so that at deployment it runs at minimum compute with no gradient machinery on board. What is stored during this slow-learning phase becomes the substance of the

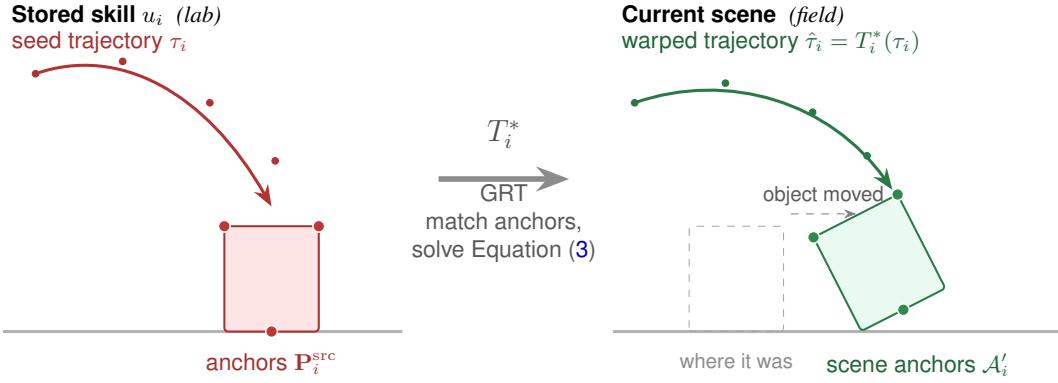


Figure 4: The Geometric Residual Transform: one stored trajectory, any object pose. A capsule stores a seed trajectory τ_i with the 3D anchors of the object it was demonstrated on (*left*); in the field the object is translated and rotated (*right*). GRT matches stored to current anchors under the task-weighted distance D_w , selects the admissible transform T_i^* , and carries the whole trajectory through it in one pass: the skill is re-aimed, not re-learned. The phase objective Φ_i verifies the emission; a failure receives a bounded local residual restricted to the failing phase. Figure 9 shows the operator on real hardware.

slow part that executes: its frozen internal parameters supply everything the deployed control cycle needs. A **fast-learning phase** is where all new learning happens in the field, encoding every competence acquired after deployment as a discrete unit in a shared memory, the way the hippocampus rapidly encodes new episodes. This learning is what happens during the fast part, and it is what grows the fast memory. The two phases learn at different rates and never interfere: new knowledge is never written back into the base. Non-interference is thus architectural: forgetting is mitigated by construction because field learning has no write path into the slow weights.

Functionally, the two parts divide the work of a deployed control cycle. The **slow part executes**: everything that must happen every cycle, at control rate, inside the edge budget. It *perceives*, it *decides*, and it *acts*, and none of this changes a single parameter. The **fast part learns**: everything that changes the system. It *selects* what is worth keeping, *stores* it one-shot and gradient-free, *re-applies* it whenever the situation recurs, and *maintains* the store within a fixed memory budget. The division is strict in both directions: the slow part never learns, and the fast part never acts on the world directly; it only modulates what the slow part is about to do.

The formulation also informs what the parts must contain. Because the slow part carries the full deployed control cycle, it must span the sensor-to-action pipeline, which gives it the form of **three cortices**: a *sensor cortex* that lifts multimodal input into stable, 3D-grounded geometry (perceive); a *reasoning cortex* that decomposes tasks into skills and judges their outcomes (decide); and an *action cortex*, a geometric skill model that predicts a stored skill’s trajectory in the current scene and executes each skill (act). Because the fast part must capture, in one write, everything a single field event teaches, its unit must bind the perception and the action content of that event to the situation that triggered it, and it must be compact, inspectable, and self-contained so that competence can be audited or removed. Section 5 details this realization: the three slow-learning cortices and the fast-learning memory they read from, whose unit is the Competence Capsule.

5 The CFAM Robotics Architecture: Slow-Learning Cortices and Fast-Learning Memory

The formulation of Section 4 motivates an end-to-end architecture, not a single module. A CFAM powers a physical asset the way a vision-language-action (VLA) policy does, and can be read as a specialization of the VLA robotics stack. We organize the deployed system as its four functional pieces (Figure 1): the **Sensor cortex** supplies the 3D-grounded embedding; the **Reasoning cortex** sequences a task plan over the capsule field and serves as the outcome oracle; the **Action cortex** emits each planned skill as one geometrically warped emission; and the **new-learning layer** creates, refines, migrates, and consolidates skill competence over the deployment lifetime under a single update law, the Continual Field Update Rule (CFUR). The capsule field $\mathcal{F}$ is the shared fast-learning memory, whose unit is defined in Section 5.4: it is read by the three runtime cortices at execution time and written only by the new-learning layer. This section details the three cortices (we use *cortex* and *module* interchangeably) and then the fast-learning memory they read from. Among the four pieces, the **Action module** (unit-of-inference shift, Section 5.3) and the near-edge extension path of **the new-learning layer** (unit-of-learning shift) are this paper’s novel, empirically evaluated contributions; Section 6 covers that extension path, test-time growth, and compression. Correction from detected field failures, open-world novelty, field-time perception writes, and language-to-trajectory synthesis are outside this paper’s scope (Section 8.2).

5.1 Sensor Cortex: ScatterNet Hybrid Deep Learning

The Sensor cortex’s purpose is to *see*: it converts raw multimodal inputs (vision, audio, LiDAR, tactile) into the two products the rest of the stack consumes—a discriminative embedding $\mathbf{z} \in \mathbb{R}^D$, binarized to the capsule field’s retrieval address $\mathbf{h} = \text{bin}(\mathbf{z})$ (Section 5.3), and per-modality sensor-event signals δ_{sensor} (tactile/force spikes, LiDAR proximity alerts, audio anomalies; the tactile arrays are shown in Figure 7).

One stored trajectory, stretched to the extremes

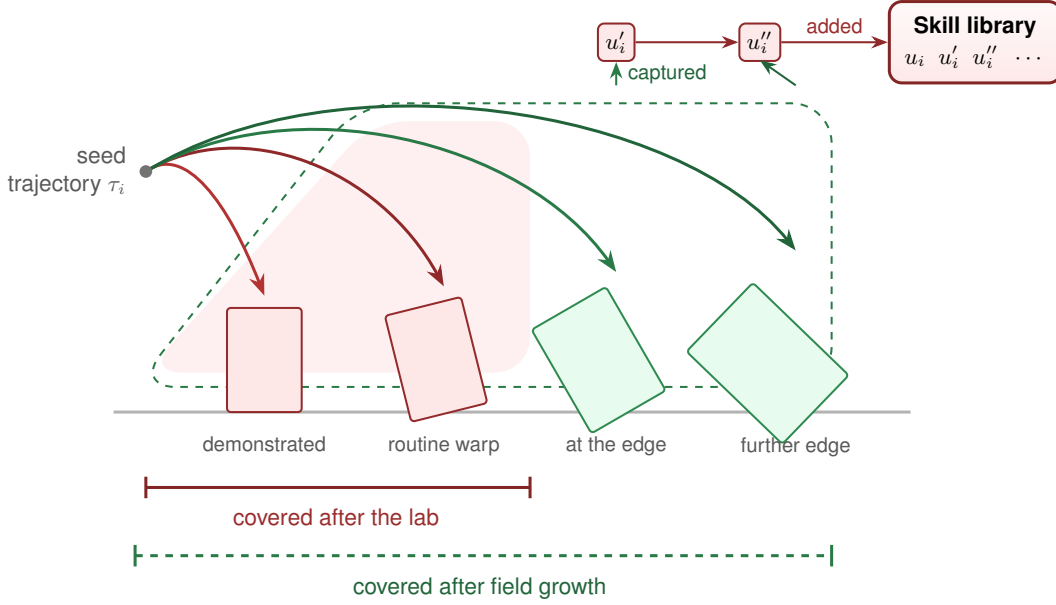


Figure 5: Generalization beyond the extremes: the envelope grows with use. GRT re-aims one stored seed trajectory at whatever pose the object takes, so a single skill covers a band of situations around the demonstration (pink). Each success at the edge of the envelope is captured as a new Competence Capsule (u'_i , u''_i), so the covered region expands with use (dashed), with no gradient step and no failure oracle. Section 6.1 formalizes the capture rule; Section 7.3 measures the resulting rise.

It adopts the **ScatterNet Hybrid Deep Learning (SHDL)** architecture [4, 5, 6]: a semi-supervised, brain-inspired encoder that learns efficiently. Its three stages mirror the $V1 \rightarrow V2/V4 \rightarrow IT$ pathway of the visual cortex (a fixed wavelet edge stage [63], an unsupervised closed-form mid-level stage, and a small supervised classifier [77]), so only the final stage needs labels—giving classification performance comparable to deep CNNs from substantially smaller training sets [4, 5]—and the fixed front-end keeps embeddings, and hence capsule addresses, stable under lighting, viewpoint, and pose variation. The same three-stage template applies to every modality, one architecture rather than a separate encoder per modality.

The load-bearing interface choice is that *the Sensor cortex outputs 3D geometry, not pixels*: downstream of SHDL the scene is lifted into a 3D representation (point clouds from RGB-D, stereo, or LiDAR) from which task-relevant anchors (object centroids, grasp points, contact points, bottleneck waypoints) are extracted. Both the embedding z that indexes $\mathcal{F}$ and the current-scene anchor set $\mathcal{A}'_i$ that drives GRT live downstream of the same geometric lift, which is what makes the Action module’s policy tractable (Section 5.3).

5.2 Reasoning Cortex: A Mission-Tuned VLM with Two Roles

The Reasoning cortex’s purpose is to *understand*: it interprets the task stated verbally, infers the user’s intent, plans at high level by decomposing the task into a sequence of stored skills, and serves as the oracle that judges whether each executed skill achieved its goal.

In detail, it is a single vision-language model $\mathcal{R}$ playing two roles inside the stack: capsule sequencer and outcome oracle. $\mathcal{R}$ is custom-trained for mission-critical operations before deploy-

ment and then frozen: CFAM requires no planner that learns in the field. In the deployed product configuration $\mathcal{R}$ is mission-tuned; in all experiments reported here $\mathcal{R}$ is an off-the-shelf frozen Qwen2.5-VL-7B (Section 7.1), so no result below depends on mission tuning. The “CFAM Reasoning” row of Table 8 is this same frozen model operating inside the CFAM stack, prompted at run time with the capsule descriptors of Equation (4); the gap to the raw Qwen2.5-VL-7B rows measures what the capsule descriptors add to the sequencer, not a different model. The mission tuning supplies the domain vocabulary and task structure of the target operations; what $\mathcal{R}$ is never trained on is the skill library itself. A planner that learns task structure during deployment violates non-iterative updates and fixed memory, and a planner trained against a lab-time library could not anticipate the post-deployment skills it will eventually have to sequence over, so $\mathcal{R}$ reads capsule descriptors at run time instead: in our implementation, $\mathcal{R}$ is a small VLM prompted with the capsule descriptors written at capsule-creation time.

Role A: capsule sequencer. Given a task instruction L and a current observation o_t , $\mathcal{R}$ reads, for every capsule, its situation key $\mathbf{h}_i$ (a field of u_i) together with its descriptor tuple $d_i = (\text{tag}_i, \text{pre}_i, \text{post}_i; \dots)$ written at capsule creation time (Equation (4)), and emits an ordered plan

$$\pi_L = (u_{i_1}, u_{i_2}, \dots, u_{i_M}), \quad u_{i_m} \in \mathcal{F} \cup \{\text{SYNTH}\}, \quad (1)$$

where each entry is either a capsule retrieved from the field or the symbol SYNTH marking a library miss at that step. The Action module then executes one capsule per plan step under GRT (Section 5.3). $\mathcal{R}$ is a read-only query layer over $\mathcal{F}$, not part of CFUR, and does not modify capsules.

Role B: outcome oracle. At a phase boundary $\mathcal{R}$ produces a binary verdict on whether the executed skill achieved its post-condition, $J_t = \text{VLM}(o_t, L) \in \{0, 1\}$. Two judgments are kept apart: CFUR writes are gated by the sensor-grounded outcome predicate Φ_i^{succ} of Section 5.3, never by J_t ; J_t is used only for semantic task progression and high-level post-condition evaluation, and the success rates of Section 7 are scored independently of both. Two routing questions follow: does the field contain a covering capsule for this step (otherwise the sequencer emits SYNTH), and is the covering capsule performing adequately (otherwise per-phase diagnostics route to refinement). $\mathcal{R}$ additionally emits bounded trajectory edits when a warp alone cannot reach the goal; because those edits act on the Action cortex’s output, they are covered with execution (Section 5.3).

The **Action cortex** that retrieves, warps, and executes the skill library is detailed next (Section 5.3); the shared fast-learning memory it reads from, the Competence Capsule and the field it forms, follows (Section 5.4), and the growth and compression that run alongside are the subject of Section 6.

5.3 Action Cortex: A Geometric Skill Model for Execution

Once the task is understood, the Action cortex turns sensor information into action: it reads the plan π_L from the Reasoning module, retrieves the corresponding capsules from $\mathcal{F}$, and emits one geometrically warped skill per phase onto the 3D scene the Sensor cortex supplies (the capsule’s stored fields read here are defined in Section 5.4).

Geometry in, skills out. The Action module reads the 3D interface of Section 5.1, the embedding $\mathbf{z}$ and the current-scene anchor set $\mathcal{A}_i$, not raw pixels, and its online optimization is over the skill’s geometric transform T_i^* and a bounded phase/control residual $\Delta\theta_i$: low-dimensional, in physically meaningful coordinates, which is what makes deployment-time skill warping tractable on edge hardware (the full tractability argument is in the supplementary material). Execution is per-skill, not per-step: a task is decomposed into physical phases (approach, align, grasp, lift, transport, place), and each phase is realized by retrieving a callable capsule from $\mathcal{F}$ and emitting the warped skill in one deterministic forward pass—one GRT pass plus the clamped blend below, at $O(K_{\text{kp}} \cdot d_{\text{net}})$ cost, where K_{kp} is the number of matched 3D anchors (keypoints) of the skill and d_{net} the width of the Sensor cortex’s embedding (D in Section 5.1), with no diffusion steps and no per-control-step decoding. A task that a frame-by-frame VLA decodes in T control steps therefore needs only $M \ll T$ capsule emissions: the number of neural action-decoding calls per task drops from T to M , independent of state-space size and reward structure (Section 3); the low-level controller still closes its loop at every one of the T control steps, but no network is decoded there.

Selecting and blending the matching capsule(s). The plan π_L settles *which skills, in what order* (Section 5.2); retrieval settles *which stored capsule(s) realize each step* in the current scene. For each step, the capsule field $\mathcal{F}$ is queried read-only,

within R4’s fixed compute and memory envelope: the current observation is matched by similarity against stored situation keys, the closest capsules within the regime-specific support radius are selected, and when several nearby capsules apply their stored values are combined under a bounded (*clamped*) blend, weighted by proximity and confidence. No learning happens here (Section 5.4 covers how the stored values are acquired).

Retrieval runs two passes. A perception pass shifts the embedding $\mathbf{z}$ to a corrected address $\mathbf{z}'$ by a clamped, confidence-weighted sum of the stored perception values, pulling toward the perception anchor $\mathbf{z}_i^*$ when the encoder itself is unreliable; an action pass then re-queries memory at $\mathbf{z}'$ and combines the stored action values the same way into an additive estimate $\mathbf{a}_{\text{add}}$ (base output plus blended residuals) and an anchor estimate $\mathbf{a}_{\text{anchor}}$ (blended stored actions). Both sums act on per-control-step action vectors, never on stored trajectories: for each active capsule the anchor pair $(\tau_i, \mathcal{A}_i)$ is first warped into the current scene by GRT (Equation (3)), and the warped emission’s action at the current phase step is the “stored action” the anchor sum combines, while the residual sum combines the $\Delta\mathbf{a}_i$ directly; the bounded-authority statement therefore compares action vectors of the same dimensionality, and the largest single stored value is the largest warped per-step action among the active capsules. This per-step blend is a fixed-cost vector operation applied to an emission that GRT has already produced for the whole phase; it is not the per-control-step network decode that a frame-by-frame VLA performs. The base output $\mathbf{a}_{\text{base}}$ entering Equation (2) is likewise not decoded per control step: the base emits its action for the whole phase once (π_0 emits an action chunk; the in-house prior emits a phase-length chunk), Equation (2) blends within that chunk at each step, and the confidence signal that gates the blend and the capture trigger is computed once per phase from that emission. Each clamped sum divides by $\max(1, \sum w_i q_i)$, with w_i decaying with Hamming distance from the query address, so a combination of several capsules can never overshoot the largest single stored value (bounded authority). The output blends the two estimates under a confidence gate:

$$\mathbf{a}_{\text{out}} = \alpha_{\text{action}} \mathbf{a}_{\text{add}} + (1 - \alpha_{\text{action}}) \mathbf{a}_{\text{anchor}}, \quad (2)$$

where α_{action} is computed from the base policy’s confidence (output entropy, logit spread, or an auxiliary head): $\alpha_{\text{action}} \approx 1$ in CORR (pure additive residual), decreasing toward the anchor-dominant blend in EXT as base reliability drops, recovering the per-regime storage behavior of Section 5.4. Because both gates measure the base’s *self-agreement*, not correctness, the independently evaluated phase objective Φ_i , which reads perceived and proprioceptive quantities rather than the model’s own estimate, is the backstop against a confidently wrong base. The full per-pathway blending and gate equations are in the supplementary material. In this paper the stored perception fields are captured directly from the demonstration at build time.

Warping and executing a skill: GRT. The **Geometric Residual Transform (GRT)** warps a selected skill onto the current physical scene, acting as a *geometric skill model* (not a learned model of environment dynamics): it predicts the skill’s full spatial trajectory $\hat{\tau}_i$ before execution and checks it against the phase

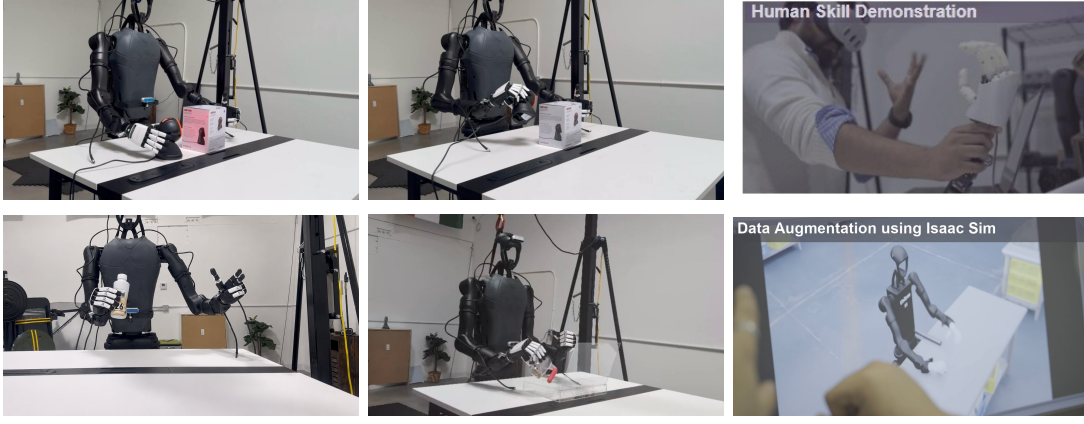


Figure 6: Building the library in the lab. Seed skills are acquired from human demonstration on the bimanual humanoid, then augmented in simulation (Isaac Sim, bottom right); each demonstration is encoded into one Competence Capsule by a single one-shot write (Section 5.4).

objective Φ_i (Figure 4; real-hardware deployments in Figure 9). For a capsule with stored anchors $\mathcal{A}_i$, seed trajectory τ_i , and admissible transform family $\mathcal{T}_i$, GRT selects

$$T_i^* = \arg \min_{T \in \mathcal{T}_i} D_w(T \mathcal{A}_i, \mathcal{A}_i') + \lambda \Omega(T), \quad (3)$$

$$\hat{\tau}_i = T_i^*(\tau_i), \quad \text{executed iff } \Phi_i^{\text{adm}}(\hat{\tau}_i, o) \leq \varepsilon_i^{\text{adm}},$$

where $\mathcal{A}_i'$ are the current-scene 3D anchors, D_w is a task-weighted, skill-conditioned anchor discrepancy, and $\Omega(T)$ charges unnecessary transform complexity (preferring the simplest admissible family, rigid $\text{SE}(3)$ by default). The first line is the geometric alignment: for the default rigid family it is solved in closed form by weighted Procrustes, and the similarity, affine, and non-rigid families use their corresponding solvers. Two predicates are kept apart. Φ_i^{adm} is the *admissibility* check on the candidate before it moves (collision margin, reachability, joint limits, contact geometry), the second line above; Φ_i^{succ} is the sensor-grounded *outcome* check on the executed trajectory (contact and force, closure, tracking tolerance, stability), which cannot be known before execution and is the only predicate that gates a CFUR write (Section 6.1). Both are evaluated from onboard sensing alone, so no VLM sits in the loop; together they are the Φ_i of Equation (4). The pipeline is therefore three-stage: weighted Procrustes generates the *candidate* transform, Φ_i^{adm} admits it, and after execution Φ_i^{succ} scores it; a candidate that fails admissibility, or an execution that fails a phase, receives a bounded local residual $\Delta\theta_i^*$ ($\|\Delta\theta\| \leq \varepsilon_i$) restricted to the failing phase—if alignment succeeds but grasp fails, only grasp is updated, with no gradient through the frozen VLA. The predicates are not hand-coded per skill. They are instantiated automatically at Build from the demonstration itself: the phase-objective template of the supplementary material (goal, alignment, contact, obstacle, smoothness, joint, and vision terms with phase-specific weights) is filled in from the demonstrated trajectory’s phase segmentation and the sensed contact, force, and tracking statistics of the demonstration, which set the active terms and their tolerances, and a field-written capsule inherits the predicate of the capsule it was warped from. A new skill family therefore needs a demonstration, not a predicate author, and “no operator label” holds at Build as well as in the field. The number of distinct predicate instances and their false-positive and false-negative rates are not reported (Section 8.2). For lo-

comotion, aerial, and wheeled skills the same operator applies with different anchors: the anchor set is the terrain, waypoint, and obstacle frames of the task (the task frames of the supplementary material), the seed trajectory is the skill’s body or base trajectory expressed in those frames (footstep and body-height profile for a gait phase, waypoint path for a flight segment, path and speed profile for a wheeled traverse), and Φ_i^{succ} is the corresponding stability or tracking predicate; what GRT re-aims is the skill’s trajectory in its task frames, not an object. The novelty is not geometric alignment per se but GRT as a continual, phase-conditioned, task-weighted, bounded-authority operator inside a persistent skill memory (Section 3 positions it against movement primitives, trajectory transfer, keypoint affordances, and residual policy learning); its execution-time guarantees, *locality* and *bounded authority* (Section 5.4), hold by construction, with proofs, full instantiations, and the transform-family hierarchy in the supplementary material.

Reasoning-mediated trajectory editing. GRT absorbs variations a stored capsule can reach under its admissible transform family (rigid $\text{SE}(3)$ for object pose and orientation; similarity $\text{Sim}(3)$ where scale changes; affine for mild deformation) but not variations that require topological change to the trajectory itself: reaching around an occluder, raising the wrist over a height obstacle. For these, the reasoning model emits an edit on top of the warped trajectory, drawn from a bounded library of waypoint operations (detour, raise, dwell) parameterized by the obstacle geometry $\mathcal{R}$ extracts from o_t and clamped to a per-step authority limit analogous to Equation (2)’s bound, so the trajectory deviation stays bounded even when the edit is wrong. An edit that succeeds twice on related scenes is written back into $\mathcal{F}$ as a new capsule via the standard CFUR path, so the skill that needs the edit is acquired over time rather than re-derived by $\mathcal{R}$ on every encounter. The edits themselves are exercised in Figure 9(c)–(d).

Library miss. When $\mathcal{R}$ cannot map a step of L to any capsule in $\mathcal{F}$, the plan emits SYNTH at that position.

The capsules executed this way are *validated competences*, each demonstrated in the lab or captured from a verified test-time success, so no exploration mechanism is required or used



Figure 7: The tactile modality of the Sensor module. Per-finger and palm tactile arrays during grasps of a bottle, a ball, and a hex key, with live per-tactel response maps; these sensor events (δ_{sensor} : contact, slip, and force spikes) are inputs to failure detection.

in this paper. The full execution model (skill-capsule view, task frames, bottleneck-state chaining, phase objectives) is developed in the supplementary material. The fast-learning memory all of this reads, and the new-learning layer writes, is defined next.

5.4 The Fast-Learning Memory: The Competence Capsule

The three cortices above execute, but none of them learns: per the formulation of Section 4, everything learned in the field is encoded into the fast-learning memory they read from, never overwriting what the base already holds (Figure 1). That memory’s single shared unit is the **Competence Capsule (CC)** (Figure 3). One capsule stores one local competence element of a skill—a *perception* correction (a shift $\Delta \mathbf{z}$ to the Sensor embedding) and an *action* correction (a residual $\Delta \mathbf{a}$ on the executed skill), bound to the situation $\mathbf{h}$ that triggers them—and the system generalizes from these one- or few-shot writes. The Sensor and Action sides read their respective slices, the Reasoning module reads capsule descriptors to compose plans (it never writes), and a single one-shot write path (CFUR) is the sole writer. All capsules live in one place, the capsule field $\mathcal{F}$, held on edge hardware within a fixed memory budget.

Where capsules come from: building the library. *Before deployment*, the library is *built* in the lab: an operator provides a few demonstrations of each target skill (teleoperation, kineshetic guidance, or a handful of successful rollouts), and each demonstration is encoded by the one-shot write, with no gradient descent over the frozen model. A demonstration covers a whole task; it is segmented at its phase boundaries (Figure 8) into the task’s two to four skills, and each segment becomes one capsule, so “one demonstration per task” yields one Build capsule per skill phase of that task. A skill is not limited to one capsule: as its envelope grows, further capsules of the same skill are added at its edge (Figure 5, u_i, u'_i, u''_i), so “one capsule” means one local competence element, and a skill is in general represented by several. The stored geometry then lets a single demonstration generalize across object poses and scenes, warped by GRT (Section 5.3). *After deployment*, the same write extends the library on-device whenever new information is identified at the edges of stored competence (Section 6.1).

Every capsule carries eight fields (seven numeric and one regime tag), plus a descriptor tuple written once at creation:

- **Situation key** ($\mathbf{h}_i$): a binary code that determines when this capsule activates. Similar scenes produce nearby keys, enabling local generalization.
- **Perception correction** ($\Delta \mathbf{z}_i$): shifts the Sensor module’s embedding toward the correct region.
- **Perception anchor** ($\mathbf{z}_i^*$): the encoder’s embedding captured during the supervised correction event, a fallback for when the encoder is completely unreliable.
- **Action correction** ($\Delta \mathbf{a}_i$): the residual added to the frozen model’s output.
- **Action anchor** ($\mathbf{a}_i^*$): the action executed during the capture event, stored as the seed trajectory τ_i together with the 3D anchors $\mathcal{A}_i$ it was recorded against; this is the pair GRT reads in Equation (3), and it is the fallback for when the base policy’s output is unreliable.
- **Persistence** (κ_i): how long the capsule survives without reactivation.
- **Confidence** (q_i): how strongly the correction is applied. Confidence grows with successful reuse.
- **Adaptation regime** ($\psi_i = (\psi_i^{\text{percep}}, \psi_i^{\text{action}})$): a pair of per-side tags, each in $\{\text{passive}, \text{CORR}, \text{EXT}\}$ (a novelty tag is reserved for future use). Each component governs its side’s support radius, generalization bandwidth, and lifecycle, with the design prior $\rho_{\text{CORR}} < \rho_{\text{EXT}}$: a corrective update stays tightly bounded around the situation that evidenced it, while an extension update is granted wider generalization bandwidth (the radius–capacity tradeoff behind this prior is analyzed in the supplementary material). The two sides can differ (e.g., perception in EXT, action in CORR).

The **canonical state** of a Competence Capsule is:

$$\begin{aligned} u_i &= (\mathbf{h}_i, \Delta \mathbf{z}_i, \mathbf{z}_i^*, \Delta \mathbf{a}_i, \mathbf{a}_i^* \equiv (\tau_i, \mathcal{A}_i), \kappa_i, q_i, \psi_i), \\ d_i &= (\text{tag}_i, \text{pre}_i, \text{post}_i; \mathcal{T}_i, \Phi_i), \end{aligned} \quad (4)$$

where the action anchor $\mathbf{a}_i^*$ carries the seed trajectory τ_i and its anchors $\mathcal{A}_i$ (so Equation (3) and Figure 4 read only capsule fields), and d_i is the descriptor tuple: the skill tag and pre-/post-conditions read by the Reasoning cortex in Equation (1), the admissible transform family $\mathcal{T}_i$, and the phase objective Φ_i used by GRT. Descriptors are metadata fixed at creation, outside the eight fields that the blend, decay, and consolidation rules act on.

Section 6 covers what runs alongside execution: test-time growth and compression of this library.

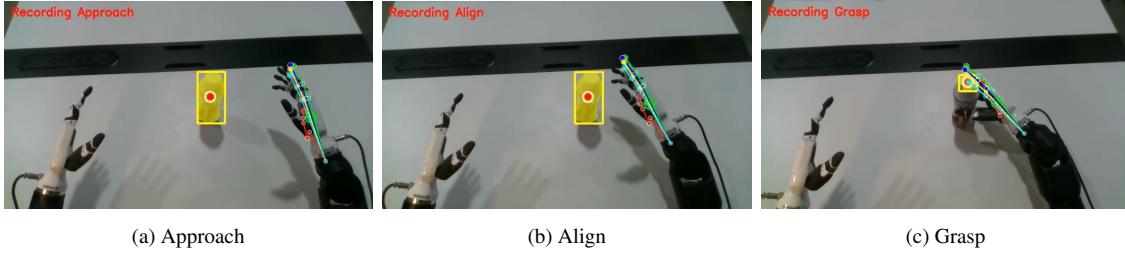


Figure 8: One skill, executed phase by phase. A stored manipulation skill on the bimanual humanoid: (a) approach, (b) align, (c) grasp, with the detected object (yellow box) and live hand-pose keypoints overlaid; each phase objective Φ must pass before the next begins (Section 5.3).

6 Test-Time Growth: Learning from the Extremes

With the library built in the lab and the execution machinery of Section 5.3 in place, this section covers what changes as the system runs: odd, near-edge cases are *identified* from signals the system already computes, *added* to the capsule field as new capsules, *generalized* from, as new points from which GRT can warp, and *compressed* so the library stays within its fixed memory budget. Acquiring a genuinely new skill when none in the library applies (open-world novelty) is outside this paper’s scope.

6.1 Growth: Adding Skills at the Extremes

Growth captures *extremes*: situations at the edge of a stored skill’s range, where the base policy’s generalization is starting to fall off (Figure 5). This is not a failure to be detected: the base is still operating, so the trigger is not an oracle but two signals the system already computes, a drop in the base policy’s *confidence* (higher output entropy, a smaller logit margin, the same signal that gates blending, Section 5.3) and a rise in the *retrieval distance* (the situation key lies in the outer margin of the nearest covering capsule’s support radius); together they mark the edge of the envelope. When a warp lands at such an edge and still *succeeds*, its outcome predicate Φ^{succ} verified, it is written back on-device as a new capsule by the one-shot write path (CFUR), so the library grows to cover that extreme and the same situation is met directly next time rather than re-warped from afar: an *EXTENSION* in the sense of Section 2. The autonomy rests on an error the system computes for itself. The base’s own uncertainty and the memory’s distance-to-support together say that this sample sits at the edge of what is stored; Φ^{succ} says that the warped execution nevertheless succeeded within a known family, which is what makes the sample relevant rather than noise. What the write stores is not a new action (the old capsule already reached it) but a persistent new local support point: the next warp starts from this point rather than from the original demonstration, so the envelope the skill can reach moves outward with each capture (Figure 5). A sample that recurs inside an existing capsule’s support does not create a new capsule: the trigger does not fire, and the verified reuse replenishes that capsule’s confidence and persistence instead, so repeated experience strengthens the same capsule; only when captures accumulate densely around one key does consolidation merge them (Section 6.2). Stated as the update rule, CFUR on this path is: *trigger* when the base confidence falls below a

threshold and the Hamming distance of the situation key from its nearest covering capsule lies within a fixed margin below that capsule’s support radius (the sample is at the edge but still covered, so a warp from that capsule exists); *verify* the executed emission with Φ^{succ} on every phase (Equation (3)), with J_t playing no part; *write* a new capsule with key $\text{bin}(\mathbf{z}_t)$, the executed warped trajectory and current anchors as $\mathbf{a}^*$, the residual $\Delta \mathbf{a}$ as the difference between the executed action and the base output, the perception fields carried over from the capsule that was warped, regime $\psi = (\text{passive}, \text{EXT})$, and initial confidence and persistence values; on every later verified reuse, *replenish* $q_i \leftarrow \min(1, q_i + \eta_q)$ and $\kappa_i \leftarrow \min(1, \kappa_i + \eta_\kappa)$; *decay* persistence every cycle by Equation (5) and remove capsules below $\kappa_{\min}$; and *consolidate* when local density exceeds its threshold (Section 6.2). A second, rarer write trigger is the reasoning-mediated edit of Section 5.3, written back after two verified successes on related scenes through the same write step. No existing capsule and no base weight is modified by a write. Genuine novelty (no stored skill of the right kind, where confidence and retrieval distance alone cannot separate a capturable extreme from a real breakdown) requires failure detection and is outside this paper’s scope. The library therefore compounds as it runs: every capsule added widens the region the Action module can warp into, so novel situations are increasingly met by warping a nearby skill in few, often zero, new shots; this mechanism predicts the rising post-deployment trajectories of Figure 11. Each warp is one forward pass, and retrieval runs over a fixed memory budget, so the per-emission cost is bounded by the configured budget rather than growing without limit as the library fills.

Growth only: no training, and a coverage-routed decision.

Neither network is trained: the slow-learning base stays frozen, and the fast-learning memory changes only by gaining new capsules, so *growth is the only learning happening here*—continual learning in CFAM is the accumulation of the fast memory. Decision-making is likewise not a joint optimization over the two networks but a routing by coverage, in the manner of complementary learning systems: when the current situation falls inside the support radius of a stored capsule, the fast path drives (the capsule supplies the skill and its corrections, confidence-weighted and clamped, Section 5.3, with the base contributing only the reference output); when no capsule covers the situation, the slow path acts alone.

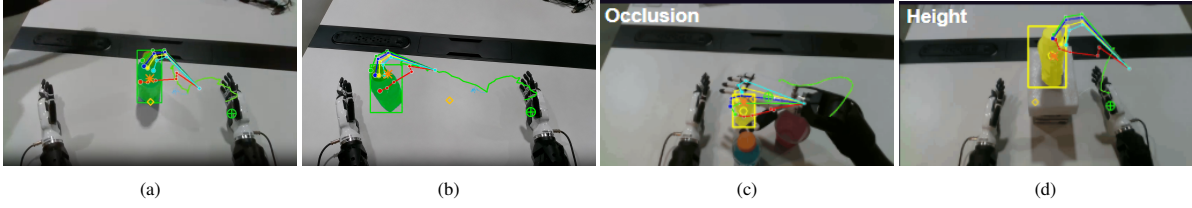


Figure 9: GRT on real hardware: one stored skill, any scene. Four deployments of the *same* stored grasp capsule: (a) nominal pose, (b) target displaced, (c) occluded among clutter, (d) raised to a different height, the latter two engaging the reasoning-mediated edit library (detour, raise) on top of the warp. Overlays: detected target, matched anchors, grasp point, bottleneck state, and end-effector trail. The capsule is stored once; per scene, GRT recomputes the transform and emits the warped trajectory in one shot, no retraining between panels.

6.2 Compression: Pruning and Consolidating

The library is held within a fixed memory budget, so growth is balanced by compression. When many capsules cluster around a single core skill, say a dozen near-identical *pick* variations, they are consolidated into one representative, while capsules that go unused over time decay in salience and are removed. Coverage therefore rises without the stored count growing unbounded, and the most useful skills stay sharp. This keeps ongoing capsule creation, in the lab and in the field, inside a fixed budget (R3), running on-device on the same CFUR schedule.

Decay. Persistence decays geometrically:

$$\kappa_i \leftarrow \gamma_{\psi_i} \kappa_i, \quad (5)$$

where $\gamma_{\psi_i} = \max(\gamma_{\psi_i}^{\text{percep}}, \gamma_{\psi_i}^{\text{action}})$ takes the slower of the two side-specific rates. Within each side, $\gamma_{\text{CORR}} < \gamma_{\text{EXT}}$: corrective fixes decay fastest because they address immediate errors, whereas extension capsules persist longer because a newly covered family member needs time to be refined (a still-slower rate is reserved for the novelty regime). Taking the max-conservative side ensures that a capsule new on either pathway is retained until its rarer side is refined. Capsules with $\kappa_i < \kappa_{\min}$ are removed. Frequently used capsules persist indefinitely because successful reuse replenishes their salience faster than decay reduces it (shown in the supplementary material); under CFUR the fixed budget prioritizes retention by reuse frequency, recency,

and coverage value, following frequency- and recency-sensitive memory allocation under limited capacity [65].

Compression. When local density exceeds a threshold, similar capsules are merged into a single representative or a small core set of representatives. Full details, together with the memory substrate’s implementation properties, are in the supplementary material.

7 Experiments

This section evaluates CFAM in four ordered stages on Skylark’s in-house multi-embodiment dataset: **(A)** prior training on D_{train} (learning-curve efficiency at fixed data fractions, with π_0 /CogACT/SpatialVLA trained on the same dataset as matched-data baselines), **(B)** held-out test on D_{test} , **(C)** autonomous test-time growth on the variation stream D_{var} , and **(D)** retention of earlier competence. This is the *Build* $\rightarrow$ *Grow* $\rightarrow$ *Retain* cycle tagged by split.

- **Evaluation domains.** Controlled synthetic benchmarks (SimplerEnv, LIBERO) support repeatable analysis, while robotic experiments provide physical validation across five assets: a Franka Panda manipulator, Unitree Go2 quadruped, Unitree H1 humanoid, quadrotor, and off-road vehicle.
- **Baseline comparisons.** On the physical platforms, the standard-policy baselines (π_0 , CogACT, SpatialVLA) are trained on the same in-house D_{train} (matched-data); on SimplerEnv and LIBERO they are the public policies; adaptation baselines (LoRA, MemoryVLA, CronusVLA) update the same standard policies after Build.
- **Four-stage protocol (A/B/C/D).** Every quantitative result below sits inside one of four stages, and every CFAM number in a caption or sentence carries the tag (split, stage, prior/backbone).
 - **Stage A — Prior training on D_{train} :** CFAM’s slow-learning cortices are trained on the Skylark in-house multi-embodiment dataset (2.6 million+ trajectories across the five robotic platforms); on the physical platforms the standard-policy baselines are trained on the same dataset (matched-data), while in simulation CFAM’s capsule field sits over a public backbone (π_0 for SimplerEnv, SpatialVLA for LIBERO). *Learning-curve evaluation:* for each training fraction $f \in \{20, 40, 60, 80, 100\}\%$, CFAM and the matched-data standard policy are trained using only $f D_{\text{train}}$.

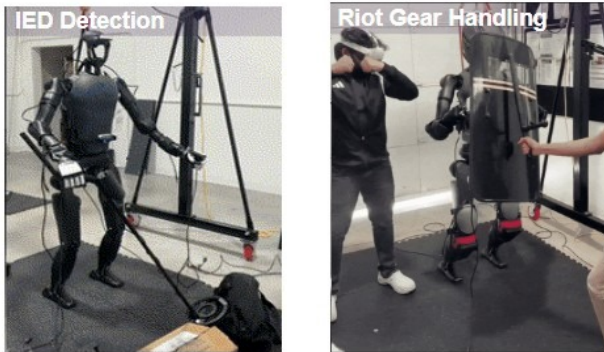


Figure 10: Test-time execution in the field. Representative executions on the humanoid platform: IED recognition (the REC skill exercised here on the humanoid; the metal-detector IED-sweep mission of Table 3 runs on the quadruped) and protective-gear handling. Slightly out-of-distribution variants remain within the same skill family but change the physical arrangement—for example, a new combination of bag and box locations that requires a different maneuver to reach a shield, or a heavier shield that changes the verified grasp-carry execution.

Table 1: Training datasets and evaluation protocols across synthetic and robotic settings. D_{train} is the trajectory count used to train the frozen prior (a public backbone in simulation, the in-house multimodal prior on the five physical platforms); it is not per-task CFAM supervision. D_{test} is the held-out Stage-B test protocol, given as scored trials *per seed* and the number of seeds (the column sums to 1,784 scored trials per seed; rates are macro-averaged over task-condition cells and then over seeds, Section 7.1); the Stage-A learning curve is scored on a separate fixed held-out set D_{curve} , disjoint from D_{train} and from D_{test} (Section 7). D_{var} is the variation stream on which autonomous test-time growth is measured (Figure 11); each row lists the number of controlled axes (viewpoint, object position, orientation, lighting, occlusion) and the number of held-out variation conditions (a base task under one variation setting, not a trial count); $|D_{\text{var}}|$ is 544 scored trials per seed across the eight rows, about 68 per row on average, which is 23.4% of the 2,328 scored trials per seed of D_{test} and D_{var} together. Each task contains two to four skills; the missions of Table 3 are longer chains (three to six skills) built from the same skill library.

Type	Dataset / platform	Embodiment	Tasks	D_{train}	D_{test}	D_{var} (axes / conditions)
Synthetic	Bridge	WidowX	4	60,096	96 trials, 3 seeds	4 / 13 conditions
	Fractal	Google Robot	5	$\sim 130\text{K}$	540–1,500 trials, 3 seeds	5 / 17 conditions
	LIBERO	Franka Panda	40	2,000	800 trials, 3 seeds	4 / 11 conditions
Robotic	Arm	Franka Panda	10	1.1M+	200 trials, 3 seeds	5 / 19 conditions
	Dog	Unitree Go2	4	87K+	48 trials, 3 seeds	4 / 14 conditions
	Humanoid	Unitree H1	2	1.2M+	40 trials, 3 seeds	5 / 16 conditions
	Drone	Quadrotor	3 sites	294K+	30 trials, 3 seeds	4 / 18 conditions
	Ground-vehicle	Off-road vehicle	3	65K+	30 trials, 3 seeds	5 / 12 conditions

Each resulting checkpoint is evaluated on the same fixed held-out set D_{curve} , which is disjoint from D_{train} ; no D_{curve} example is used for training or capsule construction, and the same tasks, trial counts, seeds, and success metric are used at every fraction. D_{test} is a separate held-out evaluation used only for Stage B. The CFAM–standard-policy gap on D_{curve} at each fraction defines Gain_{pre} .

- **Stage B — Held-out test D_{test} :** held-out task executions on the same platforms, disjoint from D_{train} and scored under the trials $\times$ seeds protocol of Table 1 (a fixed trial protocol per platform, not a fraction of D_{train}), evaluated after Build and before autonomous growth (Table 2); $\text{Gain}_{\text{test}}$ is the CFAM–standard-policy gap. CFUR test-time writes are enabled only in Stage C and in the sequential suite of Section 7.4; in Stage A, Stage B, and the LIBERO ablations they are disabled, so every capsule present there was written at Build.
- **Stage C — Variation stream D_{var} :** a further split of 544 scored trials per seed, 23.4% of the 2,328 scored trials per seed of D_{test} and D_{var} together (Table 1), produced by controlled variations of the originals (viewpoint, object position, orientation, lighting, occlusion). CFAM does a few-shot Build, then captures verified near-edge cases (Tables 3 to 5); the static standard policies fall along the stream (Figure 11); adaptation baselines receive the same D_{var} exposure under their own update mechanism. $\text{Gain}_{\text{post}}$ is the CFAM–baseline gap.
- **Stage D — Retention:** in the sequential simulation suite, after each new environment, earlier environments are re-evaluated (Table 6, Figure 12, §7.4); the physical platforms are not re-evaluated on D_{test} after Stage C. Per-platform D_{train} and D_{test} sizes appear in Table 1.
- **Catastrophic forgetting.** Sequential experiments test whether CFAM retains earlier competence as new environments and skills are introduced.

Results organization. Section 7.1 defines the baselines, metrics, and evidence boundary. Sections 7.2–7.5 then present Stage-A/B (Build and D_{test}), Stage-C (post-deployment growth on D_{var}), Stage-D (retention), and ablations, respectively.

7.1 Evaluations and Baselines

This subsection establishes the common experimental frame used throughout the results. It introduces the evaluation domains, explains the two comparison classes, and defines the criteria and evidence boundary used to interpret later results. Table 1 organizes the synthetic and robotic protocols by embodiment, build input, test scale, and task properties.

Synthetic evaluations. Controlled benchmarks isolate four questions: initial manipulation competence at Build, continued competence acquisition through Grow across a sequential environment stream, retention of earlier competence as new environments are introduced, and component-level behavior under ablation. SimplerEnv provides the controlled, sequential, and retention settings, while LIBERO provides the ablation and fine-tuning-complementarity setting. Table 1 distinguishes the Build input from the evaluated tasks and their controlled variations for each synthetic suite.

Real-world evaluations. Physical experiments test the same learning claims at increasing levels of difficulty: initial Build competence on single-phase Franka tasks (Real General, ten tabletop tasks) and continued Grow through mission chains across five physical assets. These experiments establish whether the measured gains persist outside simulation and across embodiments; retention is measured in the sequential simulation suite only (Section 7.4). Table 1 identifies the embodiment, demonstration budget, test scale, and task properties for each physical platform.

Prior and baseline configurations. CFAM’s fast-learning capsule field sits over a frozen slow-learning prior. On the five physical platforms of Tables 2 to 5 that prior is our in-house multimodal model, trained on the in-house D_{train} ; on the simulation benchmarks it is a public backbone (π_0 for SimplerEnv, SpatialVLA for LIBERO). Two Franka testbeds are exceptions and use the public π_0 backbone: the ten-task Real General suite of Section 7.2 and the sequential testbed of Section 7.5; numbers from them are always tagged as such. In every physical-platform table, π_0 , CogACT, and SpatialVLA rows are those policies

trained on the same in-house D_{train} (matched-data baselines), not the released public policies. The Reasoning cortex $\mathcal{R}$ is the off-the-shelf frozen Qwen2.5-VL-7B [78] in every experiment (Section 5.2). Adaptation-based (LoRA) and memory-based (MemoryVLA, CronusVLA) methods provide the post-deployment comparisons of Table 5; retrieval-prompting alternatives (RAG, VLM failure-prompt) are discussed qualitatively in Section 7.3.3. *Aggregation*: every reported rate is macro-averaged across task-condition cells and then averaged across the three seeds; consequently the percentages are not constrained to integer-success increments of the aggregate rollout count, and the per-seed scored-trial counts of Table 1 are the denominators of the underlying cells, not of the reported rate.

Evaluation criteria. The analysis combines task success or accuracy and one-shot/few-shot efficiency with forward and backward transfer, forgetting, component ablations, and the source of observed gains. These measurements respectively test initial competence, continued learning without catastrophic forgetting, and the contribution of the component pathways.

Evidence boundary. Autonomous near-edge growth here means operator-free capture of a physically verified, successful within-family extreme after the supervised few-shot Build. A failed execution with no usable prior skill still requires an available corrective or successful trajectory. All reported growth remains within known skill families; open-world novelty injection and later field-learning results are outside this paper.

7.2 Build: Initial Competence

Evaluation question. What competence does one demonstration buy at deployment, and how does that operating point compare with standard build baselines?

This subsection measures the competence available at the end of the supervised Build stage, before automatic field growth begins. Synthetic results establish the controlled manipulation operating point on SimplerEnv (π_0 backbone) and LIBERO (SpatialVLA backbone), while robotic results test one-shot acquisition across task families and the five physical embodiments (in-house prior).

Rate of learning. We measure prior-training data efficiency on the fixed Stage-A learning-curve split D_{curve} , not on D_{test} . At every training fraction, both CFAM and the matched-data standard policy (the in-house-trained π_0) are evaluated on the same D_{curve} protocol. At 100% of D_{train} , the standard policy reaches 52.2% on D_{curve} ; CFAM reaches the same operating point using 40% of D_{train} (52.3%), corresponding to $2.5\times$ fewer prior-training trajectories. At 100%, CFAM reaches 74.6%, a +22.4 pp advantage on this Stage-A evaluation, and the gap widens with data rather than saturating (from +11.3 pp at 20%). These values should not be compared numerically with the Stage-B D_{test} results: D_{curve} and D_{test} are independent evaluation sets with different task and condition composition. On D_{test} , the corresponding five-platform means are 56.1% for

π_0 and 65.8% for CFAM, a +9.7 pp gap (Table 2). Data efficiency compounds with the Stage-B and Stage-C gains reported below.

7.2.1 Synthetic Build Results

Across the two benchmark-level summaries, CFAM reaches 77.7% on SimplerEnv (π_0 backbone) and 81.8% on LIBERO (SpatialVLA backbone; this is the Full CFAM configuration of Table 7, with CORR and EXT capsules both written at Build and test-time writes disabled). Within SimplerEnv, performance is 76.4% on Bridge and 78.9% on Fractal. Relative to the frozen π_0 backbone, these are gains of 8.0 and 7.5 percentage points. The modification is the same in every row: one demonstration per task, segmented into one capsule per skill phase (Section 5.4), while the frozen backbone remains untouched.

7.2.2 Real-World Build Results

Per-task one-shot creation (Franka, Real General suite, public π_0 backbone). The one-shot write is measured on physical hardware on the Franka ten-task Real General suite with the public π_0 backbone: one task demonstration is segmented into its skill phases and encoded as one capsule per skill, with no gradient step. Across the ten tasks, per-task gains range from +10 pp (push, 72 $\rightarrow$ 82) to +22 pp (insert, 36 $\rightarrow$ 58), a +16.4 pp suite average (53.8 $\rightarrow$ 70.2), with large improvements on the contact-rich stack and pour tasks (+20 pp each) where the base model is systematically miscalibrated. The full per-task table is in the supplementary material. Three Franka base/CFAM pairs appear in this paper and are three different measurements: this suite (public π_0 , 53.8 $\rightarrow$ 70.2), the Real Arm column of Table 2 (the in-house prior on D_{test} , 68.6, against matched-data baselines at 59.2–61.5), and the Franka sequential testbed of Section 7.5 (public π_0 , 55.8 $\rightarrow$ 71.6). The in-house prior is a separate model from the public π_0 (Section 7.1); the two are never mixed within a table. The gains come from both slices of the capsule: the perception-pathway studies in the supplementary material isolate the perception correction’s independent contribution (−6.1 pp when Δz is zeroed on LIBERO) and show the two slices address genuinely distinct failure types.

Across the real-world datasets. Table 1 widens the lens from the Franka suite to five platform-specific datasets. Each uses the same one-demonstration CFAM Build operation, but the embodiment, task family, action space, and test protocol differ.

In-house prior scale. The in-house multimodal prior was trained on more than 2.6 million trajectories across the five physical embodiments, 22 task and site settings, and 79 enumerated variations spanning manipulation, legged locomotion, aerial inspection, and off-road navigation (Table 1); these are prior-training trajectories, not per-task CFAM supervision—each robotic Build still uses one demonstration per evaluated task. The resulting CFAM Build success on D_{test} ranges from 59.7% on the quadraped to 71.5% on the quadrotor inspection setting, with the contact-rich humanoid setting at 60.8% and

Table 2: Comparison of VLA models with CFAM on the held-out test split D_{test} : CFAM after few-shot Build, before autonomous test-time growth. The CFAM row uses the public π_0 backbone for the SE-Bridge and SE-Fractal columns and the in-house multimodal prior for the five robotic columns. In the five robotic columns the π_0 , CogACT, and SpatialVLA rows are those policies trained on the same in-house dataset as matched-data baselines (not the released public policies); in the SE-Bridge and SE-Fractal columns they are the released public policies (trained on their own public data), and the π_0 row there is the same frozen backbone the CFAM row sits on.

Method	SE-Bridge	SE-Fractal	Real Arm	Real Dog	Real Humanoid	Real Drone	Real Vehicle
CogACT	51.3	68.1	61.5	54.5	52.5	55.2	56.3
SpatialVLA	42.7	75.1	60.4	50.5	49.6	48.9	45.6
π_0	68.4	71.4	59.2	52.3	55.0	58.5	55.6
CFAM (Ours): π_0 backbone (sim) / in-house prior (real)	76.4	78.9	68.6	59.7	60.8	71.5	68.5

a 65.8% cross-platform mean (Table 2); the post-deployment growth from this Build anchor is evaluated in Section 7.3.

Across five physical platforms. The same one-shot write generalizes across the five physical platforms (Table 1): CFAM leads the matched-data π_0 baseline on every one (+5.8 to +13.0 pp) and the strongest matched-data baseline on every one (+5.2 to +13.0 pp; per-platform values in Table 2). A single demonstration written as a capsule, with the deployed weights untouched (preserving the non-interference guarantee of Section 4), outperforms three standard policies trained on the full D_{train} . Qualitatively, the humanoid’s gains concentrate on contact-rich manipulation where the in-house prior is most miscalibrated, and the quadruped’s on unstable terrain (gravel, slip recovery) where a single demonstration of the recovery gait warps across the terrain family. The load-bearing claim is that the *capsule machinery is form-factor-agnostic*, not that skills are shared across form factors: the same GRT and the same write-grow-consolidate dynamics govern every library; only the embedding pipeline and action dimensionality change.

Component-level pathway comparisons. The Build evaluation decomposes CFAM into reasoning, perception, and the downstream action pathway. Detailed reasoning and perception pathway comparisons at Build are provided in Section A (Tables 8 and 9); the action pathway comparison is reported here as the downstream execution outcome.

7.2.3 Build-Method Comparison

The controlled build comparison is summarized by Table 2. On SimplerEnv, one-shot CFAM exceeds every public static policy on both splits with no per-task GPU adaptation (LoRA is compared on D_{var} in Table 5, not at Build). On the physical platforms, one-shot CFAM leads the strongest matched-data baseline by +5.2 to +13.0 pp; the capsule representation converts one demonstration into a reusable, geometrically grounded unit of competence without touching the deployed weights. This subsection covers base competence building with standard supervised methods only; comparisons with post-deployment adaptation and memory methods are part of the field-learning analysis of Section 7.3.

7.3 Autonomous Near-Edge Test-Time Growth

Evaluation question: after a few-shot supervised Build, can the deployed system autonomously expand competence from verified near-edge experience?

This subsection evaluates post-deployment growth separately in simulation and on physical systems. The synthetic stream measures whether forward transfer remains positive as five new environments arrive in sequence; the real-world stream measures the increase from the deployment-day Build anchor to the verified pre-novelty Grown endpoint across skill families, assets, and mission chains. Retention of earlier competence is analyzed separately in Section 7.4.

7.3.1 Synthetic Growth Results

CFAM maintains positive forward transfer in every synthetic environment (+11.5 to +14.2 pp; +12.8 pp on average; per-environment values in the FT columns of Table 6), even as the stream introduces new objects, layouts, and task types. This is the synthetic growth result: the deployed memory adds useful competence at each stage without a gradient update to the frozen π_0 prior. Section 7.4 uses the same stream to test whether those additions disturb earlier environments.

7.3.2 Real-World Growth Results

Growth and generalization are not only mechanisms of Sections 5.4 and 6; on the near-edge deployment stream they are measurable results. At deployment the system warps stored skills onto the current scene. When confidence and retrieval distance mark a successful execution at the edge of a stored skill’s support, the verified execution is written back as an extension capsule. This capture is autonomous because no operator selects, labels, or demonstrates the event and no gradient step changes the frozen prior; it is bounded because only verified near-edge extensions of known skill families are included in this paper.

Across the stream, the action side’s per-skill success rises 74.0 → 87.9% (Table 4): execution rises as the warpable envelope expands. Tables 8 and 9 report the reasoning and perception pathways at Build. The Build column of Table 4 is the one-shot library scored on D_{test} before growth, and the Grown column is the same library scored on D_{test} after the D_{var} stream has been traversed with writes enabled; at the skill-family level the pair therefore separates GRT generalization alone from GRT plus accumulated captures, on conditions the system never captured from.

Compression during growth. Consolidation carries performance weight as well as footprint: removing it costs 2.3–3.0 pp (Section 7.5).

Table 3: Master mission table: ten mission-critical scenarios (two per asset), each an ordered skill chain over $\mathcal{F}$. **Build** = the supervised one-shot library at deployment; **Grown** = the pre-novelty endpoint after automatic capture of verified within-family extremes, i.e. after the whole Stage-C stream (Standard = the static standard policy under the same mission protocol; † marks the mission plotted in Figure 11, whose Build, $x=100$, and Grown, $x=160$, values are its anchors). Prefixes: L- = LOCO-, M- = MANIP-, REC = recognize.

Asset	Mission	Ordered skill chain	Mission accuracy (%)			
			Standard Build	CFAM Build	Standard Grown	CFAM Grown
Humanoid	Hazardous-object recovery (firearm)	REC → L-legged-flat → M-reach → M-align → M-grasp → M-carry	52.2	66.3	13.4	79.2
	Protective-gear donning (shield)†	REC-gear → M-reach → M-grasp → M-don/wear → PA-posture-stance	48.0	62.1	10.0	75.8
Robot Dog	Metal-detector IED sweep	L-legged-rough → M-carry (payload) → REC-anomaly → PA-mark-target → L-slip-recovery	47.8	68.9	6.1	77.5
	Confined-space structural recon†	L-legged-rough → L-gap-cross → REC-hazard → PA-mark-target	50.0	71.1	10.0	81.4
Drone	Perimeter recon & track	L-aerial-hover → L-aerial-waypoint → REC-target → PA-loiter-track	56.5	75.0	21.1	83.7
	Post-blast aerial damage survey†	L-aerial-hover → L-aerial-waypoint → REC-damage → PA-orbit-inspect	53.0	71.5	18.0	80.6
Arm	Precision part insertion (ordnance)	REC-part → M-reach → M-align → M-grasp → M-place → M-insert	54.9	68.7	17.9	79.9
	Render-safe / wire-cut (EOD)†	REC-assembly → M-reach → M-align → M-grasp (fine) → M-cut	53.0	66.8	15.0	77.0
Ground Vehicle	Off-road approach to objective	L-wheeled-terrain → REC-obstacle → route/avoid	57.4	77.6	26.9	85.4
	Convoy follow & checkpoint halt†	L-wheeled-terrain → L-wheeled-follow → REC-checkpoint → M-precise-stop	55.0	75.2	26.0	84.5
Overall			52.8	70.3	16.4	80.5

Mission-level reading. Table 3 reports the same arc at mission level (plotted per asset in Figure 11): mean accuracy rises from 70.3% at Build to 80.5% at the pre-novelty Grown endpoint. The three Grown readings of this section are three different units and are reported separately rather than as nested readings of one measurement: per-skill success on D_{var} (Table 4, 87.9), per-platform task success on D_{var} after growth (Table 5, 68.0 mean over the five real platforms), and per-mission accuracy (Table 3, 80.5); the three readings are therefore not nested and cannot be compared as a product of per-skill rates. The largest gains occur in the contact-rich arm and humanoid chains, matching manipulation’s largest family-level gain in Table 4; those families leave the greatest gap after one-shot build. Qualitatively, the field cases of Figure 13 illustrate the kind of near-OOD condition the stream contains (familiar objects in a new spatial arrangement, a heavier carried load, dense vegetation and a narrow passage on a grass-trained route, an unseen payload disturbance on a moving quadruped). They are slightly out-of-distribution changes inside an existing mission family, not unrelated tasks.

Table 4: Robotic growth by skill family on D_{var} . Build = the one-shot library scored on D_{test} after Build, before growth; Grown = the same library scored on D_{test} after the D_{var} stream has been traversed with writes enabled (the verified pre-novelty endpoint). Δ is Grown minus Build success rate. Overall row is the unweighted mean across the five family rows.

Skill family	Build (%)	Grown (%)	Δ (pp)
Legged locomotion	74.4	86.8	+12.4
Aerial	77.6	88.8	+11.2
Wheeled	73.5	89.0	+15.5
Manipulation	72.2	88.3	+16.1
Other perception-action	72.5	86.6	+14.1
Overall (unweighted mean)	74.0	87.9	+13.9

7.3.3 Comparison with Post-Deployment Adaptation and Memory Methods

The following comparison places CFAM’s growth against the methods that also use information after deployment: gradient/adaptor, memory-buffer, and retrieval-prompting approaches. Quoted literature results are not treated as matched controlled comparisons.

For a deployed policy that must improve after it ships, there are two natural architecture alternatives: the *gradient/adaptor route* (write the new competence into the weights: LoRA, full fine-tuning) and the *memory-buffer route* (attach an episodic store the policy reads at inference: MemoryVLA, CronusVLA). Table 5 compares CFAM with VLA augmentation and adaptation approaches on the variation stream D_{var} . Against the memory-augmented systems on D_{var} , CFAM leads on both SimplerEnv splits (+12.0 to +13.0 pp over MemoryVLA, +11.9 to +26.5 pp over CronusVLA) and exceeds MemoryVLA on the pooled robotic suite by +13.5 to +22.1 pp per platform (Table 5). We attribute this performance to the regime-governed capsule structure: the two-regime memory organization enables qualitatively different storage for corrections and extensions. Against gradient-based adaptation, CFAM exceeds LoRA on both SimplerEnv splits and every robotic platform while requiring *no retraining of the frozen model*; the demonstration budgets differ only at Build (one demonstration per task for CFAM against ten per task for the LoRA fine-tune), after which every method receives the same D_{var} exposure. Neither alternative keeps learning: the gradient route stops when the fine-tuning budget is spent (and forgets when it resumes, Section 7.4), and the buffer route accumulates entries without consolidating them, whereas the capsule library keeps growing at test time.

Retrieval-prompting alternatives: RAG and failure-state prompting. A third alternative uses no new machinery at all:

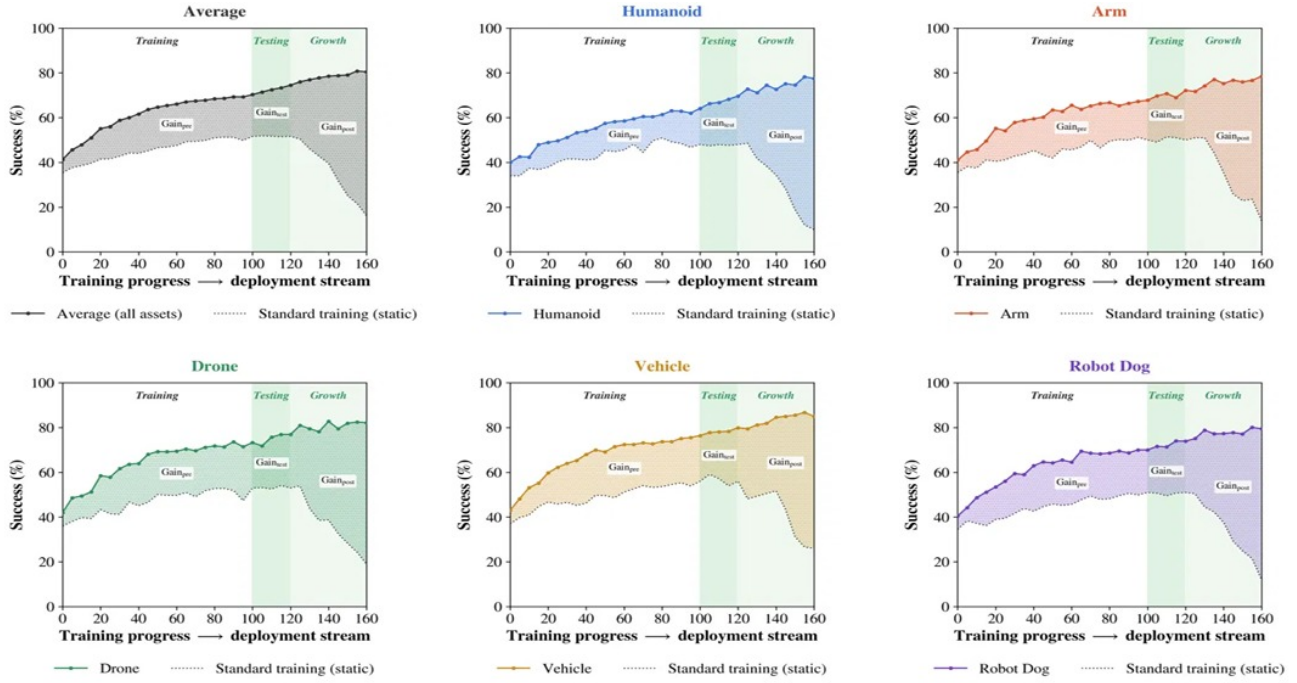


Figure 11: CFAM versus the standard policy across the deployment stream, per real-world embodiment and averaged. Solid = CFAM; dotted = the standard (static) policy. In the *Training* region (0–100, Stage A) CFAM learns faster, rising above the standard policy at every fraction of D_{train} . In the *Testing* region (100–120, Stage B) CFAM sustains that advantage on the held-out test. In the *Growth* region (120–160, Stage C) CFAM keeps growing as verified near-edge cases are captured, while the standard policy falls. The pattern holds on every embodiment and on the average; each panel shows one mission, marked $\dagger$ in Table 3; its Build ($x=100$) and Grown ($x=160$) values are the anchors, and the Training region is the same mission scored at training checkpoints (Average: 70.3 $\rightarrow$ 80.5).

keep the frozen policy and its Reasoning VLM, and put the adaptation burden on the prompt. These alternatives are discussed qualitatively and are not scored in Table 5. *VLM failure-prompt* feeds the failure state back to the Reasoning VLM at the next attempt with no persistent store; *RAG* persists every failed-then-corrected episode as a retrievable document and prepends the top- k matches at inference. The failure modes are informative. First, corrections must round-trip through language: continuous geometric detail (a grasp offset of a few centimeters, a wrist angle of a few degrees) is lost in verbalization, which is precisely the content a capsule stores as a numeric residual and GRT re-applies geometrically. Second, retrieval-prompting requires a VLM decode on *every adaptation step*, whereas CFAM invokes the Reasoning model once per skill phase and not at all per correction. This makes the capsule path less dependent on repeated language-model inference during adaptation. In capsule terms (Section 5.4), RAG is a CC with its perception and action slices collapsed to text: the situation key survives as embedding similarity, but the numeric correction payload,

bounded authority, and consolidation are lost.

Absolute performance context. CFAM’s SimplerEnv-Fractal result is competitive with recent gradient-retrained policies on the same suite despite performing no retraining: representative reported values are 60.5% for Dream-VLA [79] and 63.0% for OpenVLA-OFT [80] under visual matching. We do not claim a like-for-like win (protocols and fine-tuning data differ across these reports, and our figures come from the configuration of Section 7.1), but the gradient-free operating point is not paid for in absolute success on this suite. The architectural cost of refusing to update the base is instead visible in the ceiling of Table 4: one-shot Build reaches roughly 84.2% of the Grown endpoint (74.0/87.9), and test-time growth closes the remainder.

Table 5: Comparison of adapted policies with CFAM after adaptation on the variation stream (D_{var}); the metric is per-platform task success (%) on D_{test} , the same metric and split as Table 2. Each adaptation method receives the same D_{var} exposure under its own update mechanism (LoRA fine-tune, memory writes), and the comparison is head-to-head on the same split: CFAM leads the strongest adaptation baseline in every column, by 11.9 to 20.8 pp. The CFAM row is above its own Build values in Table 2 in every column, by 1.8–2.4 pp on the five real platforms and by 0.8–1.5 pp on the SimplerEnv splits, so the two tables are a before/after pair.

Method	SE-Bridge	SE-Fractal	Real Arm	Real Dog	Real Humanoid	Real Drone	Real Vehicle
<i>Adaptation and memory baselines on D_{var}</i>							
CronusVLA	51.4	67.8	54.5	39.8	42.9	48.7	41.8
MemoryVLA	64.9	67.7	57.3	48.4	45.6	51.8	48.4
π_0 + LoRA	65.3	63.6	52.6	46.7	49.2	53.1	50.8
CogACT + LoRA	61.2	64.4	53.3	49.3	49.9	50.1	50.4
CFAM (ours): 1-shot Build + test-time growth	77.9	79.7	70.8	61.9	63.2	73.9	70.3

Table 6: Continual adaptation across five sequential simulated environments. SimplerEnv task families, π_0 backbone; mean over 3 seeds. FT = forward transfer; BT = backward transfer; Forget = prior tasks with $> 5\%$ success drop.

Stage	CFAM		LoRA		MemVLA	
	FT	BT	FT	BT	FT	BT
Env 1	+14.2	—	+16.8	—	+8.3	—
Env 2	+12.8	−0.3	+14.1	−4.7	+7.1	−1.2
Env 3	+11.5	−0.5	+11.3	−8.9	+6.4	−2.8
Env 4	+13.1	−0.4	+9.2	−13.6	+5.8	−4.1
Env 5	+12.4	−0.6	+7.8	−18.2	+5.1	−5.7
Avg	+12.8	−0.5	+11.8	−11.4	+6.5	−3.5
Forget	2.1%		34.7%		10.8%	

7.4 Growth Without Forgetting

Evaluation question: does adding new competence preserve performance on earlier environments?

The central architectural claim, that the memory grows without disturbing what the base or the earlier memory already holds, is tested head-on by sequential-environment adaptation: five simulated environments constructed from the SimplerEnv task families [81] (π_0 backbone) are encountered in succession, each introducing new objects and layouts within the known task families relative to its predecessors; after adapting in each, the agent is re-evaluated on all earlier ones. Table 6 shows the result (Figure 12 plots the retained fraction after each stage): near-zero backward transfer (−0.5 pp) and minimal forgetting (only 2.1% of prior tasks lose more than 5 pp) across five sequential environments, consistent with the locality guarantee and the no-forgetting invariant (both stated and proved in the supplementary material). LoRA suffers catastrophic forgetting (34.7% of prior tasks lose more than 5 pp; BT −18.2 pp by Env 5) because it modifies shared parameters. CFAM’s forward transfer remains stable (+11.5 to +14.2 pp), with no measurable degradation attributable to saturation of the fixed-budget capsule field $\mathcal{F}$ over this five-environment horizon. Retention is measured in the sequential simulation suite.

The no-interference principle, in its simplest form. The near-zero forgetting above is not peculiar to CFAM’s machinery; it follows from a structural property CFAM shares with the simplest frozen-backbone classifier, nearest-class-mean (NCM) [60, 61, 62]: because each stored unit occupies a separate slot and enrolling a new one changes no existing parameters, prior knowledge is preserved by construction rather than by a regularizer. CFAM generalizes this primitive along the axes a deployed policy needs—a capsule stores a per-situation action and perception correction rather than a per-class mean, is addressed over a support radius, weighted by confidence, typed by regime, warped by GRT, and consolidated under a fixed budget—while NCM is the degenerate case (one passive, radius-zero capsule per class, no warping, no growth). The −0.5 pp backward transfer of Table 6 is this principle measured on a sequential suite.

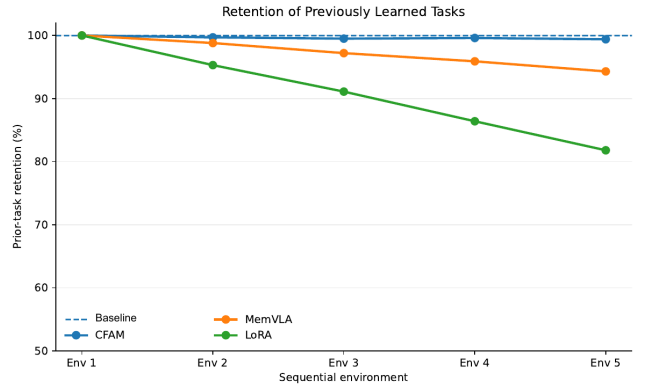


Figure 12: Retention of previously learned tasks across the sequential suite. The curve shows competence retained after each successive environment (dashed: perfect retention). CFAM remains within 0.6 pp of perfect retention, while MemVLA drifts to 94.3% and LoRA falls to 81.8%.

7.5 Ablations

Evaluation question: which memory, control, geometric, and consolidation components account for the observed gains?

Ablations use the SpatialVLA backbone and report average success rate across the four LIBERO suites under conventional static-set evaluation.

Per-regime and architectural ablation. Table 7 isolates each component and reports per-suite numbers. Each pre-novelty regime contributes independently: corrective +6.9 pp, extension +5.3 pp (Full CFAM = CORR + EXT; the CORR capsules are written at Build from the supplied demonstrations, as in every experiment here, and no novelty-regime capsules are used anywhere in this paper). The full combination (+16.1 pp over base) exceeds the sum of the two isolated contributions (6.9 + 5.3 = 12.2 pp), indicating an interaction between the regimes: a correction capsule and an extension capsule active on the same query recover cases that neither recovers alone. Architecturally, the capsule-field controller is the most load-bearing of the component swaps (removing it costs 8.3 pp). Dense storage costs 5.0 pp. GRT and capsule dynamics are complementary (+4.8 pp and +9.9 pp alone, +16.1 pp together).

Consolidation ablation. Removing intra-regime consolidation, merging of redundant capsules *within* a regime, costs 3.0 pp, and removing cross-regime consolidation costs 2.3 pp (Table 7). Without consolidation, capsules accumulate redundancy and retrieval signal-to-noise degrades: compression is not only a footprint mechanism but a performance one.

Complementarity with fine-tuning. A natural concern is that CFAM’s gains vanish as the base model improves. We test this by layering CFAM atop a *fine-tuned* base (LoRA, rank 32, $\alpha = 32$) on two testbeds: SpatialVLA on LIBERO-Long, and the Franka sequential tabletop testbed with the public π_0 backbone (a different suite and backbone from the in-house-prior Real Arm column of Table 2 and from the Real General suite of Section 7.2). LoRA + CFAM reaches $74.3 \pm 1.3\%$ on LIBERO-Long (SpatialVLA) and $75.1 \pm 1.5\%$ on the real-robot testbed (π_0 , Franka), outperforming both

Table 7: Component ablations on LIBERO (SpatialVLA backbone, static-set evaluation). Mean and SD are over three seeds, per suite as Spatial/Object/Goal/Long success rate (%). Δ is the change in the four-suite average against Full CFAM, the reference row; negative is worse. The blocks isolate the pre-novelty regimes (CORR, EXT), swap one architectural component at a time (storage, field controller, retrieval, GRT, capsule dynamics), and remove consolidation.

Configuration	Mean	SD	Δ
Base only	74.1/69.8/67.5/51.2	1.4/1.6/1.7/2.3	-16.1
CORR only	81.7/76.5/74.2/57.8	1.2/1.4/1.5/1.9	-9.2
EXT only	79.4/75.1/72.8/56.4	1.3/1.5/1.6/2.0	-10.9
Full CFAM	87.9/85.2/83.1/70.9	1.0/1.1/1.3/1.5	—
Dense memory	83.1/80.4/78.3/65.2	1.2/1.3/1.5/1.7	-5.0
No field controller	80.2/77.1/74.8/61.8	1.4/1.5/1.7/1.9	-8.3
Nearest-neighbor action	81.6/79.0/76.5/63.7	1.3/1.4/1.6/1.8	-6.6
GRT only	78.4/74.5/72.1/56.8	1.3/1.5/1.6/2.0	-11.3
CC dynamics only	82.7/79.8/77.2/62.4	1.2/1.3/1.5/1.8	-6.3
No intra-regime consolidation	85.2/82.4/80.1/67.3	1.1/1.2/1.4/1.6	-3.0
No cross-regime consolidation	85.8/83.1/80.9/68.0	1.1/1.2/1.4/1.6	-2.3

LoRA alone (64.7/65.2%, i.e. +9.6/+9.9 pp) and frozen + CFAM (70.9/71.6%, i.e. +3.4/+3.5 pp), from frozen bases of 51.2/55.8% (3 seeds). The gain from adding CFAM on top of LoRA is smaller than on the frozen model. This is expected: as the base model improves, fewer errors remain for the memory to correct; the remaining gains come from extension-regime capsules that address gaps LoRA cannot fill. Crucially, the LoRA + CFAM configuration requires no additional GPU compute beyond LoRA’s initial fine-tuning: capsule dynamics are gradient-free and run entirely at inference time. Practitioners need not choose between fine-tuning and CFAM: a robot can be fine-tuned once in the lab and then equipped with the memory for growth.

8 Discussion

The central result is a separation between the competence installed when a physical AI system is built and the competence it can acquire after deployment: matched-data efficiency and leadership at Build, autonomous growth on the variation stream while adaptation baselines fall behind, and near-perfect retention in the sequential simulation suite (Section 7).

8.1 What Test-Time Growth Means for Physical AI

CFAM does not keep training the frozen foundation model after deployment. It expands a separate capsule field whose entries are local, inspectable perception-action corrections. In the measured pre-novelty streams, confidence and retrieval distance identify a successful execution at the edge of a stored skill’s support, and the physical phase objective verifies that success before the execution is written back. The result is a larger reusable envelope without a gradient step on the deployed base. The rise from the Build to Grown anchors therefore measures added post-deployment competence, not merely repeated inference from a fixed library.

This is *autonomous learning from near-OOD cases at test time*. The Build library is few-shot and supervised: each installed task begins with a supplied demonstration. After that initialization, however, no operator chooses or labels the successful cases that are captured; the deployed system identifies, verifies,

and stores them from its own execution signals on-device.

What a learned near-OOD case looks like. The difference is small enough to preserve the task’s structure but large enough to sit outside the Build examples: a new bag-box arrangement forcing a different maneuver to reach the shield, a heavier shield in the same reach-grasp-carry family, a grass-trained route meeting dense weeds and a narrow opening, a moving gait meeting an unseen impulsive payload disturbance (Figure 13). CFAM learns these slightly out-of-distribution combinations by extending an applicable prior skill; they do not imply autonomous weapon selection or engagement.

The build comparisons and the growth results answer different questions: what one demonstration buys at deployment relative to the matched-data and adaptation alternatives, and whether that one-shot operating point remains fixed. Growth is bounded by the skill families the library already holds.

8.2 Limitations and Scope

Evidence boundary and duration. The evidence ends at the defined near-edge boundary: known skill families remain relevant throughout. Autonomous capture has been demonstrated only on the evaluated hardening streams and within the measured horizon; longer-duration and open-world deployment remain untested. Open-world novelty injection, failure detection, correction from detected field failures, field-time perception writes, and language-to-trajectory synthesis are outside this paper’s scope and are not evaluated here. Several quantities that bear on the claims are not measured here either. *Data and controls:* the in-house prior with an empty capsule field, the same prior fine-tuned on the single Build demonstration, and the same demonstration exploited by trajectory replay or nearest-neighbour retrieval without capsules, the controls that would isolate the capsule representation’s share of the Build gain; a per-platform Stage-C run with test-time writes disabled (the skill-family Build column of Table 4 is the only such control) and a held-out post-growth split of unseen conditions; the reachable envelope as a function of the number of captures; growth of the perception and reasoning pathways along the stream; the number of unique skills per platform; the size and composition of D_{curve} ; the per-platform D_{var} denominators and condition allocation; and the simulated (Isaac Sim) share of D_{train} per

platform. *Statistics*: per-seed dispersion, confidence intervals, and paired tests on the reported rates; and the number of distinct Φ predicate instances and their false-write and false-abstention rates. *Deployment footprint*: the footprint behind R3 and R4 (edge device per platform, whether the Reasoning cortex ran on the asset or off-board during the quadrotor and quadruped trials, GRT, retrieval, and write latencies against the control period, capsule size and counts before and after D_{var} , memory budget and occupancy, and consolidation merges and evictions during Stage C). *Protocol*: the mission trial counts, retry rules, and scoring of Table 3; whether the SpatialVLA base of Table 7 was LIBERO-fine-tuned; and the starting checkpoints, parameter counts, optimization budgets, and tuning used to train π_0 , CogACT, and SpatialVLA on the in-house dataset and to adapt MemoryVLA and CronusVLA to the non-manipulation embodiments. The architectural argument for R4 is that a write is one forward pass and one memory insertion, gradient-free; measurements of this write path in on-field deployments are future work, so the deployment claim here is stated as a property of the design.

Supervision and write quality. Autonomous near-edge extension is not open-world discovery. Build requires supplied demonstrations, and failed executions with no usable prior skill require an available corrective or successful trajectory. Moreover, the confidence and retrieval signals used to route an event are not correctness guarantees. A confidently wrong base can suppress the fallback, false-write and false-abstention rates are not yet calibrated, and a persistent wrong capsule can be reused. The independent phase objective is the principal backstop, but it verifies an execution after emission rather than guaranteeing it beforehand.

Frozen-base and representation limits. CFAM adapts around the frozen policy; it cannot repair every deficiency inside that policy. If the encoder maps physically different states to indistinguishable addresses, or if an action lies outside the base system’s sensing, actuation, and executable skill family, local residuals cannot create the missing capability. The formal properties bound authority, locality, and convergence under stated conditions; they do not guarantee task success.

Geometry and morphology. GRT depends on reliable correspondences. Occluded or mismatched anchors degrade the candidate transform, while rigid and affine warps are poorly suited to strongly deformable objects. Cross-morphology transfer requires a compatible action representation and is not established here.

Retention and capacity. The retention evidence is simulation-only (Table 6); the physical platforms have not been re-evaluated on D_{test} after Stage-C growth, so real-platform retention is untested. The no-interference result applies to disjoint supports left untouched. Consolidation can merge nearby capsules and therefore trades redundancy against fidelity. The memory budget is finite, and a sufficiently long or diverse deployment will eventually require eviction or a larger store; which knowledge

should be retained under saturation remains an open systems and governance question.

Evaluation breadth. The present experiments cover tabletop manipulation, sequential simulated environments, ten mission chains, and five physical platforms, but they do not establish generality across coordinated two-handed manipulation (the humanoid tasks of Figures 6 and 8 use one hand at a time), deformable-object handling, or multi-room navigation. These results should be read as evidence for the CFAM growth primitive and its Build $\rightarrow$ Grow $\rightarrow$ Retain sequence, not as a complete demonstration of autonomous lifelong learning.

9 Conclusion

CFAM demonstrates that the competence of a physical AI system need not be fixed at deployment. Across four ordered stages on Skylark’s in-house multi-embodiment dataset with matched-data baselines, CFAM reaches the standard policy’s operating point with $2.5\times$ less prior-training data, leads every matched-data baseline on the held-out split, autonomously captures verified slightly out-of-distribution cases on-device, raising action success by 13.9 percentage points while adaptation baselines fall behind, and, in the sequential simulation suite, retains earlier competence, with backward transfer of -0.5 percentage points versus -11.4 percentage points for LoRA.

The result does not depend on continual gradient retraining of the frozen base. New competence is written into bounded, inspectable perception–action capsules and reused through geometric warping. This gives post-deployment learning a localized form: the system can add coverage while preserving the shared substrate and the provenance of each addition.

The present evidence establishes autonomous near-edge extension, not open-world discovery; open-ended novelty, longer-duration field learning, and cross-morphology generalization remain future work (Section 8.2).

Today’s physical AI systems largely stop learning when training ends. CFAM provides evidence that they can instead continue acquiring bounded competence post-deployment.

References

- [1] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al., “OpenVLA: An open-source vision-language-action model,” in *Conference on Robot Learning (CoRL)*, 2024.
- [2] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al., “ π_0 : A vision-language-action flow model for general robot control,” in *Robotics: Science and Systems (RSS)*, 2025.
- [3] J. L. McClelland, B. L. McNaughton, and R. C. O’Reilly, “Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory,” *Psychological Review*, vol. 102, no. 3, pp. 419–457, 1995.
- [4] A. Singh and N. Kingsbury, “ScatterNet hybrid deep learning (SHDL) network for object classification,” in *IEEE International Workshop on Machine Learning for Signal Processing (MLSP)*, 2017.
- [5] A. Singh, *ScatterNet Hybrid Frameworks for Deep Learning*. PhD thesis, University of Cambridge, 2019.
- [6] A. Singh, “Continuously evolving and interactive disguised face identification (DFI) with facial key points using ScatterNet hybrid deep learning (SHDL) network.” U.S. Patent 11,594,074, 2023. Granted.
- [7] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations (ICLR)*, 2022.
- [8] P. Kanerva, *Sparse Distributed Memory*. Cambridge, MA: MIT Press, 1988.
- [9] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Chormanski, T. Ding, D. Driess, A. Dubey, C. Finn, et al., “RT-2: Vision-language-action models transfer web knowledge to robotic control,” *arXiv preprint arXiv:2307.15818*, 2023.
- [10] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, et al., “Octo: An open-source generalist robot policy,” in *Robotics: Science and Systems (RSS)*, 2024.

- [11] Q. Li, Y. Liang, Z. Wang, L. Luo, X. Chen, M. Liao, F. Wei, Y. Deng, S. Xu, Y. Zhang, X. Wang, B. Liu, J. Fu, J. Bao, D. Chen, Y. Shi, J. Yang, and B. Guo, "CogACT: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation," *arXiv preprint arXiv:2411.19650*, 2024.
- [12] J. Björck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, et al., "GROOT N1: An open foundation model for generalist humanoid robots," *arXiv preprint arXiv:2503.14734*, 2025.
- [13] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *arXiv preprint arXiv:2305.16291*, 2023.
- [14] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, "ExpEL: LLM agents are experiential learners," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, pp. 19632–19642, 2024.
- [15] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [16] Z. Liu, A. Bahety, and S. Song, "REFLECT: Summarizing robot experiences for failure explanation and correction," *arXiv preprint arXiv:2306.15724*, 2023.
- [17] G. Sarch, Z. Wu, M. J. Tarr, and K. Fragkiadaki, "HELPER: An interactive embodied agent that plans, helps, and learns," *arXiv preprint arXiv:2310.15127*, 2023.
- [18] J. Wu, R. Antonova, A. Kan, M. Lepert, A. Zeng, S. Song, J. Bohg, S. Rusinkiewicz, and T. Funkhouser, "TidyBot: Personalized robot assistance with large language models," *Autonomous Robots*, vol. 47, no. 8, pp. 1087–1102, 2023.
- [19] T. Yoneda, J. Yin, J. Fang, P. Sun, H. Zhang, and M. R. Walter, "Statler: State-maintaining language models for embodied reasoning," *arXiv preprint arXiv:2306.17840*, 2023.
- [20] H. Sun, Y. Zhuang, L. Kong, B. Dai, and C. Zhang, "AdaPlanner: Adaptive planning from feedback with language models," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [21] J. O. Zhang, A. Sax, A. Zamil, L. Guibas, and J. Malik, "Side-tuning: A baseline for network adaptation via additive side networks," in *European Conference on Computer Vision*, pp. 698–714, 2020.
- [22] R. Zhao, T. Ingebrand, S. Chinchali, and U. Topcu, "MoS-VLA: A vision-language-action model with one-shot skill adaptation," *arXiv preprint arXiv:2510.16617*, 2025.
- [23] J. Tack, J. Kim, E. Mitchell, J. Shin, Y. W. Teh, and J. R. Schwarz, "Online adaptation of language models with a memory of amortized contexts," in *Advances in Neural Information Processing Systems*, 2024.
- [24] R. Li, W. Guo, Z. Wu, C. Wang, H. Deng, Z. Weng, Y.-P. Tan, and Z. Wang, "MAP-VLA: Memory-augmented prompting for vision-language-action model in robotic manipulation," *arXiv preprint arXiv:2511.09516*, 2025.
- [25] C. Liu, Y. Liu, T. Wang, Q. Zhuang, J. C. Liang, W. Yang, R. Xu, Q. Wang, D. Liu, and C. Han, "On-the-fly VLA adaptation via test-time reinforcement learning," *arXiv preprint arXiv:2601.06748*, 2026.
- [26] Z. Bai, C. Gao, and M. Z. Shou, "EVOLVE-VLA: Test-time training from environment feedback for vision-language-action models," *arXiv preprint arXiv:2512.14666*, 2025.
- [27] T. Silver, K. Allen, J. Tenenbaum, and L. Kaelbling, "Residual policy learning," in *arXiv preprint arXiv:1812.06298*, 2018.
- [28] T. Johanning, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine, "Residual reinforcement learning for robot control," in *Proceedings of the IEEE International Conference on Robotics and Automation*, pp. 6023–6029, 2019.
- [29] W. Xiao, J. Xie, T. Zhang, H. Lin, L. Fu, H. Xue, J. Lu, Y. Yang, C. Dai, Z. Wang, J. Wu, G. Wang, S. S. Sastry, K. Goldberg, L. Fan, Y. Zhu, and G. Shi, "ENPIRE: Agentic robot policy self-improvement in the real world," *arXiv preprint arXiv:2606.19980*, 2026.
- [30] A. J. Jipspeert, J. Nakanishi, H. Hoffmann, P. Pastor, and S. Schaal, "Dynamical movement primitives: Learning attractor models for motor behaviors," *Neural Computation*, vol. 25, no. 2, pp. 328–373, 2013.
- [31] S. Calinon, "A tutorial on task-parameterized movement learning and retrieval," *Intelligent Service Robotics*, vol. 9, no. 1, pp. 1–29, 2016.
- [32] J. Schulman, J. Ho, C. Lee, and P. Abbeel, "Learning from demonstrations through the use of non-rigid registration," in *International Symposium on Robotics Research (ISRR)*, 2013.
- [33] L. Manuelli, W. Gao, P. Florence, and R. Tedrake, "kPAM: Keypoint affordances for category-level robotic manipulation," in *International Symposium on Robotics Research (ISRR)*, 2019.
- [34] P. Yu, P. Huang, C. Chawla, G. Shi, J. Li, and C. Liu, "Autonomous integration and improvement of robotic assembly using skill graph representations," *arXiv preprint arXiv:2603.12649*, 2026.
- [35] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 627–635, 2011.
- [36] A. Kumar, Z. Fu, D. Pathak, and J. Malik, "RMA: Rapid motor adaptation for legged robots," in *Robotics: Science and Systems (RSS)*, 2021.
- [37] H. Shi, B. Xie, Y. Liu, L. Sun, F. Liu, T. Wang, E. Zhou, H. Fan, X. Zhang, and G. Huang, "MemoryVLA: Perceptual-cognitive memory in vision-language-action models for robotic manipulation," *arXiv preprint arXiv:2508.19236*, 2025.
- [38] M. Lei, H. Cai, Y. Yang, Y. Wu, J. Ren, Z. Cui, L. Tan, J. Hong, G. Hu, S. Zhu, S. Jiang, G. Wang, J. Tan, Z. Wan, Z. Li, Z. Li, S. Cui, Y. Zhao, and Y. Han, "RoboMemory: A brain-inspired multi-memory agentic framework for interactive environmental learning in physical embodied systems," *arXiv preprint arXiv:2508.01415*, 2025.
- [39] E. Tulving, "Episodic and semantic memory," in *Organization of Memory* (E. Tulving and W. Donaldson, eds.), pp. 381–403, New York: Academic Press, 1972.
- [40] M. Lin, X. Liang, B. Lin, J. Liu, Z. Jiao, K. Li, Z. Yan, Y. Sun, W. Liufu, Y. Ma, J. Hu, Y. Liu, S. Zhao, Y. Zhuang, and X. Liang, "EchoVLA: Robotic vision-language-action model with synergistic declarative memory for mobile manipulation," *arXiv preprint arXiv:2511.18112*, 2025.
- [41] W. Hu, Y. Hong, Y. Wang, L. Gao, Z. Wei, X. Yao, N. Peng, Y. Bitton, I. Szepes, and K.-W. Chang, "3DLMM-Mem: Long-term spatial-temporal memory for embodied 3D large language model," *arXiv preprint arXiv:2505.22657*, 2025.
- [42] T. Kagaya, S. Lakshmi, A. Ye, J. Y. Thong, J. Karlekar, S. Pranata, N. Murakami, A. Kinose, and Y. You, "ViReSkill: Vision-grounded replanning with skill memory for LLM-based planning in lifelong robot learning," *arXiv preprint arXiv:2509.24219*, 2025.
- [43] G. Tziatas and H. Kasaei, "Lifelong robot library learning: Bootstrapping composable and generalizable skills for embodied control with language models," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [44] S. Xie, Y. Zhang, R. Wang, and X. Chen, "Uni-Skill: Building self-evolving skill repository for generalizable robotic manipulation," *arXiv preprint arXiv:2603.02623*, 2026.
- [45] P. Kanerva, "Sparse distributed memory and related models," in *Associative Neural Memories: Theory and Implementation*, pp. 50–76, Oxford University Press, 1993.
- [46] V.-P. Berges, B. Oğuz, D. Haziza, W.-t. Yih, L. Zettlemoyer, and G. Ghosh, "Memory layers at scale," *arXiv preprint arXiv:2412.09764*, 2024.
- [47] J. Lin, L. Zettlemoyer, G. Ghosh, W.-t. Yih, A. Markosyan, V.-P. Berges, and B. Oğuz, "Continual learning via sparse memory finetuning," *arXiv preprint arXiv:2510.15103*, 2025.
- [48] V. Vosylus and E. Johns, "Instant policy: In-context imitation learning via graph diffusion," in *International Conference on Learning Representations (ICLR)*, 2025.
- [49] E. Johns, "Coarse-to-fine imitation learning: Robot manipulation from a single demonstration," *arXiv preprint arXiv:2105.06411*, 2021.
- [50] D. Wang, E. Shelhamer, S. Liu, B. Olshausen, and T. Darrell, "TENT: Fully test-time adaptation by entropy minimization," in *International Conference on Learning Representations (ICLR)*, 2021.
- [51] Y. Sun, X. Wang, Z. Liu, J. Miller, A. A. Efros, and M. Hardt, "Test-time training with self-supervision for generalization under distribution shifts," in *International Conference on Machine Learning (ICML)*, pp. 9229–9248, 2020.
- [52] Q. Wang, O. Fink, L. Van Gool, and D. Dai, "Continual test-time domain adaptation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 7201–7211, 2022.
- [53] M. Zhang, S. Levine, and C. Finn, "MEMO: Test time robustness via adaptation and augmentation," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 38629–38642, 2022.
- [54] M. Jia, L. Tang, B.-C. Chen, C. Cardie, S. Belongie, B. Hariharan, and S.-N. Lim, "Visual prompt tuning," in *European Conference on Computer Vision (ECCV)*, pp. 709–727, Springer, 2022.
- [55] H. Liu, D. Tam, M. Muqeeth, J. Mohta, T. Huang, M. Bansal, and C. Raffel, "Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 1950–1965, 2022.
- [56] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, et al., "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [57] A. A. Rusu, N. C. Rabinowitz, G. Desjardins, H. Soyer, J. Kirkpatrick, K. Kavukcuoglu, R. Pascanu, and R. Hadsell, "Progressive neural networks," *arXiv preprint arXiv:1606.04671*, 2016.
- [58] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Proceedings of the 34th International Conference on Machine Learning*, pp. 1126–1135, 2017.
- [59] Defense Advanced Research Projects Agency, "Lifelong learning machines (L2M)," DARPA program, Broad Agency Announcement HR001117S0016, 2017. <https://www.darpa.mil/program/lifelong-learning-machines>.
- [60] T. Mensink, J. Verbeek, F. Perronnin, and G. Csurka, "Distance-based image classification: Generalizing to new classes at near-zero cost," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 11, pp. 2624–2637, 2013.
- [61] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, "iCaRL: Incremental classifier and pattern recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2001–2010, 2017.
- [62] P. Janson, W. Zhang, R. Aljundi, and M. Elhoseiny, "A simple baseline that questions the use of pretrained-features for continual learning," in *NeurIPS 2022 Workshop on Distribution Shifts*, 2022.
- [63] A. Singh and N. Kingsbury, "Dual-tree wavelet scattering network with parametric log transformation for object classification," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2017.
- [64] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations (ICLR)*, 2021.
- [65] A. Singh and J. L. McClelland, "Human-like learning for frequency-skewed classification," in *Proceedings of the Annual Meeting of the Cognitive Science Society (CogSci)*, 2020.
- [66] B. N. Patro and V. S. Agneeswaran, "Scattering vision transformer: Spectral mixing matters," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [67] K. Friston, "The free-energy principle: A unified brain theory?," *Nature Reviews Neuroscience*, vol. 11, no. 2, pp. 127–138, 2010.
- [68] R. P. N. Rao and D. H. Ballard, "Predictive coding in the visual cortex: A functional interpretation of some extra-classical receptive-field effects," *Nature Neuroscience*, vol. 2, no. 1, pp. 79–87, 1999.
- [69] A. Clark, "Whatever next? Predictive brains, situated agents, and the future of cognitive science," *Behavioral and Brain Sciences*, vol. 36, no. 3, pp. 181–204, 2013.
- [70] D. Kumaran, D. Hassabis, and J. L. McClelland, "What learning systems do intelligent agents need? Complementary learning systems theory updated," *Trends in Cognitive Sciences*, vol. 20, no. 7, pp. 512–534, 2016.
- [71] A. Pouget, P. Dayan, and R. Zemel, "Information processing with population codes," *Nature Reviews Neuroscience*, vol. 1, no. 2, pp. 125–132, 2000.
- [72] A. P. Georgopoulos, A. B. Schwartz, and R. E. Kettner, "Neuronal population coding of movement direction," *Science*, vol. 233, no. 4771, pp. 1416–1419, 1986.
- [73] E. R. Kandel, Y. Dudai, and M. R. Mayford, "The molecular and systems biology of memory," *Cell*, vol. 157, no. 1, pp. 163–186, 2014.
- [74] K. Friston, "Hierarchical models in the brain," *PLoS Computational Biology*, vol. 4, no. 11, p. e1000211, 2008.
- [75] E. Tulving, "Memory and consciousness," *Canadian Psychology/Psychologie canadienne*, vol. 26, no. 1, pp. 1–12, 1985.
- [76] E. Dupoux, Y. LeCun, and J. Malik, "Why AI systems don't learn and what to do about it: Lessons on autonomous learning from cognitive science," *arXiv preprint arXiv:2603.15381*, 2026.
- [77] A. Singh and N. Kingsbury, "Efficient ConvNet learning using parametric log based dual-tree wavelet ScatNet," in *IEEE International Conference on Computer Vision Workshops (ICCVW)*, 2017.
- [78] S. Bai, K. Chen, X. Liu, J. Wang, W. Ge, S. Song, K. Dang, P. Wang, S. Wang, J. Tang, et al., "Qwen2.5-VL technical report," *arXiv preprint arXiv:2502.13923*, 2025.
- [79] J. Ye, S. Gong, J. Gao, J. Fan, S. Wu, W. Bi, H. Bai, L. Shang, and L. Kong, "Dream-VL & Dream-VLA: Open vision-language and vision-language-action models with diffusion language model backbone," *arXiv preprint arXiv:2512.22615*, 2025.
- [80] M. J. Kim, C. Finn, and P. Liang, "Fine-tuning vision-language-action models: Optimizing speed and success," *arXiv preprint arXiv:2502.19645*, 2025.
- [81] X. Li, K. Hsu, J. Gu, K. Pertsch, O. Mees, H. R. Walke, C. Fu, I. Lunawat, I. Sieh, S. Kirmani, S. Levine, J. Wu, C. Finn, H. Su, Q. Vuong, and T. Xiao, "Evaluating real-world robot manipulation policies in simulation (SimplerEnv)," in *Conference on Robot Learning (CoRL)*, 2024.
- [82] OpenAI, "GPT-4o system card," tech. rep., OpenAI, 2024. [arXiv:2410.21276](https://arxiv.org/abs/2410.21276).
- [83] Gemini Robotics Team et al., "Gemini Robotics: Bringing AI into the physical world," *arXiv preprint arXiv:2503.20020*, 2025.
- [84] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," in *International Conference on Machine Learning (ICML)*, 2021.
- [85] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, et al., "DINOv2: Learning robust visual features without supervision," *Transactions on Machine Learning Research*, 2024.
- [86] S. Nair, A. Rajeswaran, V. Kumar, C. Finn, and A. Gupta, "R3M: A universal visual representation for robot manipulation," in *Conference on Robot Learning (CoRL)*, 2022.
- [87] X. Chen, S. Xie, and K. He, "An empirical study of training self-supervised vision transformers," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

A Field Deployment Imagery and Pathway Comparisons

The two ground platforms of the pre-novelty mission set (Table 3) are shown during field trials, providing the embodiment context for the slightly out-of-distribution cases discussed in Section 8. The reasoning- and perception-pathway comparisons referenced from Section 7.2 follow (Tables 8 and 9).

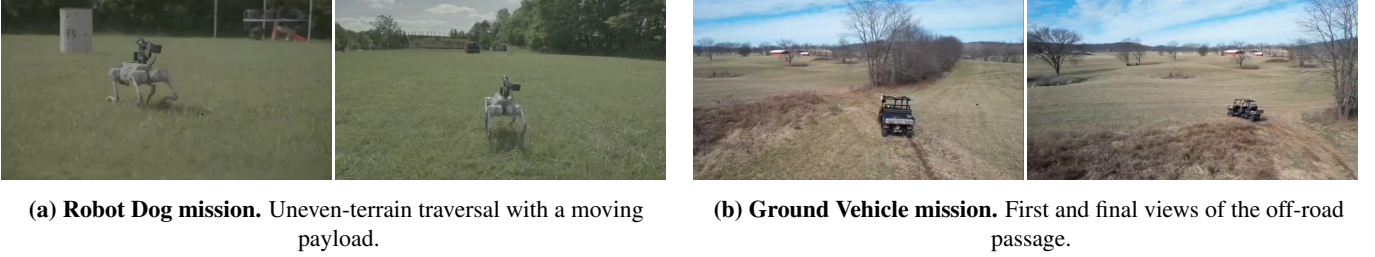


Figure 13: Field deployment imagery and concrete near-OOD cases. (a) **Robot Dog:** the legged platform provides the embodiment for extending a moving gait to an unseen impulsive or recoil-like payload disturbance while preserving stability. (b) **Ground Vehicle:** a route learned on open grass is extended when dense weeds leave only a narrow opening. These are new, slightly out-of-distribution conditions inside known skill families; the system identifies, verifies, and stores the successful extension autonomously on-device.

Table 8: Reasoning pathway comparison across simulated and real-robot suites. Success rate (%) is reported for the reasoning pathway. The CFAM row is the same frozen Qwen2.5-VL-7B as the second row, operating inside the CFAM stack and prompted with the capsule descriptors of Equation (4) (Section 5.2); the difference between the two rows is what the descriptors add, not a different model. Baselines: GPT-4o [82], Qwen2.5-VL-7B [78], Gemini Robotics-ER [83].

Method	SE-Bridge	SE-Fractal	Real Arm	Real Dog	Real Humanoid	Real Drone	Real Vehicle
GPT-4o	38.8	35.4	35.2	36.5	39.6	25.6	31.3
Qwen2.5-VL-7B	45.3	42.8	38.5	39.2	32.6	29.5	34.5
Gemini Robotics-ER	39.3	36.5	31.5	33.6	35.6	27.1	32.5
CFAM Reasoning (Ours; Qwen2.5-VL-7B in stack)	54.3	58.8	45.6	48.7	43.7	46.9	43.6

Table 9: Perception pathway comparison across simulated and real-robot suites. Success rate (%) is reported for the visual representation used by the downstream action policy. The CFAM row is the in-domain-trained SHDL Sensor cortex, whereas the baselines are frozen generic encoders. Baselines: CLIP [84], DINOv2 [85], R3M [86], MoCo-v3 [87], ViT-IN (ImageNet-supervised ViT [64]).

Perception Method	SE-Bridge	SE-Fractal	Real Arm	Real Dog	Real Humanoid	Real Drone	Real Vehicle
CLIP	61.8	68.4	61.1	43.1	46.5	47.4	42.2
DINOv2	64.7	71.2	64.3	44.8	42.3	48.7	44.3
R3M	66.9	73.5	66.7	45.1	48.2	50.8	45.6
MoCo-v3	59.8	67.1	58.3	42.3	41.1	46.3	40.2
ViT-IN	58.6	65.9	65.4	43.7	45.9	46.8	42.3
CFAM Perception (Ours)	73.2	78.6	76.8	52.1	63.4	65.5	63.8